

Dishonest-Majority Secure Computation via PIR-Authenticated Multiplication Triples

Elette Boyle
Reichman University and
NTT Research
eboyle@alum.mit.edu

Niv Gilboa
Ben-Gurion University
gilboan@bgu.ac.il

Matan Hamilis
Reichman University
matan@hamil.is

Yuval Ishai
Technion
yuvali@cs.technion.ac.il

Ariel Nof
Bar-Ilan University
ariel.nof@biu.ac.il

February 13, 2026

Abstract

We revisit the question of minimizing the overhead of security against malicious parties in dishonest-majority secure computation.

A leading approach, pioneered by the SPDZ line of protocols, uses homomorphic MACs to authenticate computation: Parties effectively compute a MAC on the computation output using *authenticated multiplication triples* (AMT). However, securely generating these AMTs presently sits as the cost bottleneck.

In this work, we introduce a new technique for enabling SPDZ-style verification via homomorphic MACs, while *bypassing the need for AMT*. We instead rely on the specific structure of state-of-the-art fast pseudorandom correlation generators (PCG) for generating standard (unauthenticated) multiplication triples (MT). Parties authenticate the computation result via an authenticated variant of *private information retrieval* (PIR), relying on the sparse representation of MT produced by these PCGs.

This opens the door to a wide range of PIR optimizations and tradeoffs from the literature, resulting in asymptotic and concrete improvements over the traditional AMT-based approach. For example, in the Boolean 2-party case with $\sigma = 40$, we get a $3\times$ to $8\times$ computation improvement (and comparable communication) over best approaches using PCG to generate AMT, where the latter exploits variants of the Stationary Syndrome Decoding assumption of Kolesnikov et al. (Crypto 2025). With $m \geq 3$ parties we obtain even larger improvements while reducing the asymptotic dependence on m from cubic to quadratic.

1 Introduction

A major challenge in the area of secure multiparty computation (MPC) is bridging the efficiency gap between protocols that only offer security against semi-honest adversaries and ones that are secure against malicious adversaries. From a concrete efficiency perspective, this question is challenging especially in the dishonest-majority setting, where efficient general compilers such as the “GMW compiler” [46] exist in theory, but do not provide a viable solution in practice.

This challenge has led to the exploration of various techniques to improve the end-to-end costs of secure computation in the dishonest-majority setting with malicious security. A particularly successful approach is to make use of *correlated randomness*. Originating from the work of Beaver [3], who showed how to use “multiplication triples” (MT) for secure arithmetic computation with no honest majority, many MPC protocols make extensive use of correlated randomness of similar form [6, 40, 48, 55, 38, 41, 39, 37, 26, 30].

Authenticated multiplication triples. Pioneered by the SPDZ line of works [6, 40], a special type of correlated randomness that enables low online communication with good end-to-end costs consists of *authenticated multiplication triples* (AMT): standard MT (one for each multiplication gate) consisting of additively shared (a_i, b_i, c_i) where $c_i = a_i b_i$, along with a sharing of the same values times a secret global multiplier. The latter effectively serve as a *homomorphic MAC* to authenticate the values produced by the online protocol. Such an authentication needs to be done over a large field, or alternatively repeated multiple times over a small field, to provide a sufficiently good level of security.

This has become a leading approach in the area, spawning a range of optimizations, implementations, and improvements, e.g. [50, 51, 37, 8, 31]. By design, in all these protocols the bulk of the work lies in the setup phase, where the AMTs are generated. In particular, the typical communication complexity associated with generating the AMTs in existing protocols is by orders of magnitude higher than the size of the circuit being evaluated.

Pseudorandom correlation generators. Motivated by these high communication costs, efficient constructions of *pseudorandom correlation generators* (PCGs) [16, 17] aim to mitigate the communication bottlenecks associated with securely generating correlated randomness. PCGs provide a means for parties to locally expand short correlated seeds into long pseudorandom instances of a given target correlation, without any communication except for the (typically cheap) distributed seed generation. Indeed, recent PCG constructions based on variants of the Learning Parity with Noise (LPN) assumption enable *concretely efficient* secure generation of many *unauthenticated* multiplication triples, with sublinear (in fact, polylogarithmic) communication, including the secure generation of the seeds [19, 12, 11, 54, 2]. This directly yields a practical, sublinear-communication preprocessing for semi-honest MPC, even in the multi-party setting.

However, these techniques do not apply as efficiently to the more complex correlation of *authenticated* multiplication triples, necessary for extending this approach to protocols with malicious security. Concretely efficient PCGs for the AMT correlation exist only in the limited setting of 2-party correlations [19, 54]. While being concretely efficient, generating authenticated multiplication triples over *small* fields still forms a painful bottleneck and incurs a significant overhead (up to a factor of $\approx \sigma$, a statistical security parameter) compared to their unauthenticated counterparts. While these limitations can in some cases be circumvented [1], this comes at an even greater asymptotic and concrete cost, making them impractical for large-scale MPC. In particular, the asymptotic communication costs of generating the triples increase from polylogarithmic to $\Omega(\sqrt{n})$ for circuit size n , and this is additionally multiplied by an $\Omega(m^3)$ factor where m is the number of parties. Using PCG-generated *pairwise* AMT in the style of “BDOZ” [6] may be feasible for arithmetic circuits and a small number of parties, but results in online communication that scales quadratically in the number of parties, which is also undesirable.

To conclude, current PCG machinery cannot be efficiently used to generate *authenticated* triples

that support an online protocol which scales linearly with the number of parties, making the multi-party (and even the three-party) case impractical. Consequently, all practical protocols for MPC in these settings require the communication complexity of preprocessing to be much bigger than the circuit size.

1.1 Our Results

We put forth a new approach for malicious security in dishonest-majority MPC, achieving SPDZ-style authentication of the computation, while *bypassing the need for authenticated multiplication triples* (AMT).

Instead, our approach reduces the task of computing a homomorphic MAC on the secure circuit evaluation to a special variant of Private Information Retrieval (PIR) [33, 53] that we call *Authenticated Sparse Linear PIR* (ASL-PIR). Informally, ASL-PIR is a malicious-secure computation of a secret sparse linear combination of a database owned by all parties. This variant of PIR, in turn, can be reduced to standard PIR and as such can benefit from the extensive literature on efficient PIR protocols.

The MAC computation via ASL-PIR is made possible by an observation on the structure of state-of-the-art constructions of PCGs for (standard) unauthenticated multiplication triples (MT). In particular, such constructions provide a *sparse representation* of the resulting MT, in the sense that the triples are computed as some public linear map G applied to a sparse vector $\vec{e}$ (where parties hold additive secret shares of $\vec{e}$). Computing the SPDZ-style homomorphic MAC at the conclusion of the semi-honest MPC computation corresponds to a further linear map G' applied to the triples: collectively, amounting to the desired MAC output $G' \cdot G \cdot \vec{e}$.

Viewed in a different manner, one can equivalently interpret this computation as a (*secret*) *sparse linear combination* (determined by the sparse vector $\vec{e}$) of entries from a *public database* induced by the mapping $G' \cdot G$. The ASL-PIR enables the parties to perform this operation directly, without the need of ever generating authenticated triples themselves.

Our resulting protocol for obtaining malicious security incurs as little as $\text{polylog}(n)$ additive communication overhead over the semi-honest baseline, where n is the size of the circuit, and scales only quadratically in the number of parties (compared to the cubic scaling of [1]) while relying purely on LPN-style assumptions. This new approach not only yields simpler protocols with better concrete efficiency, but also implies asymptotic improvements. Under LPN-style assumptions, we improve the asymptotic computational cost of practical PCG-based m -party MPC by up to a factor of $\approx \sigma$ (statistical security parameter) for $m = 2$ and $\approx \sigma m$ in general, without increasing the communication cost. In particular, we obtain the following:

Theorem 1.1 (MPC via PIR-based AMT, informal). *Assuming the LPN-style assumptions from [20, 13, 11, 54, 2] over $\mathbb{F}$ (the choice of which depends on $\mathbb{F}$) with constant rate and weight $w = w(\lambda)$, there exists an m -party MPC protocol Π_C , computing an arithmetic circuit C of size n (counting multiplication gates) over $\mathbb{F}$, with up to $m - 1$ malicious parties, such that:*

- *The end-to-end communication complexity of Π_C is equal to that of the semi-honest baseline MT-based protocol (namely, a total of $O(nm)$ field elements) plus an additive sublinear term.*
- *The end-to-end total computational cost of Π_C is $O(m^2 wn)$, asymptotically the same as that of the PCGs for (unauthenticated) MT from [20, 13, 11, 54, 2].*

The respective LPN assumptions permit for the weight sparsity parameter to be set as low as $w \in \text{polylog}(n)$, thus implying total computation of $\tilde{O}(m^2n)$.

The *end-to-end* communication of this protocol is comparable to the *online* communication in SPDZ, and much lower than end-to-end SPDZ. The protocol improves over the computational cost of the previous PCG-based SPDZ-style protocol from [1] by an asymptotic factor of m . An additional improvement factor of $\approx \sigma$ for Boolean circuits, which applies to any number of parties, can be obtained via the use of a new LPN-style assumption that we discuss below. This improvement is specific to our PIR-based authentication approach and does not apply for mitigating the efficiency bottlenecks of competing approaches, regardless of the underlying assumption.

The high-level source of the factor- m improvement over [1] is avoiding the need for a secret-shared unit vector (i.e., Distributed Point Function (DPF)) between each triple of parties, used in [1] for authentication. Instead, our computational cost is dominated by the underlying PCG for MT, which only requires a DPF between each pair of parties. The m -party PIR we use for authentication is asymptotically dominated and can be implemented without DPFs (e.g., information-theoretically with square-root communication [33]).

Concrete efficiency. From a concrete efficiency viewpoint, we believe that our new approach strikes a good balance between the different cost measures, significantly improving over all previous approaches even in the 2-party case and even more so in the multi-party case. In the 2-party case with Boolean circuits, our approach improves over the state-of-the-art PCG-based SPDZ protocol [54] by a factor of $3\times$ to $8\times$ in overhead above the semi-honest protocol, breaking the inherent $\sigma\times$ computational overhead incurred by authenticating binary MT. For $m > 2$ parties, our approach offers an even bigger improvement compared to existing approaches. Even on the first non-trivial case of $m = 3$ parties, our approach improves compared to the state-of-the-art PCG-based SPDZ protocol [1] by $3\times$ to $33\times$ in different settings while breaking the $\sigma\times$ overhead barrier even in that setting, while any PCG-based SPDZ protocol inherently incurs a $\sigma m\times$ overhead. For example, for $n = 3^{20}$ AND gates, the computation of [1] would require over 147 hours, while our approach takes 44 hours and with further optimizations as little as 4.5 hours. We note that our comparison focuses against solutions in the “sublinear-communication” setting, as protocols which do not enjoy this property (including those in the standard SPDZ line) have even further slowdowns. As reported by estimates of the TinyOT protocol [42], generating the same number of triples AMTs would take over 155 hours.

At a high level, the computational cost of our protocol is dominated by several invocations of PIR on public databases. We further investigate two optimizations for *batching* these PIR queries, taking advantage of the regular query structure: (1) leveraging the recently introduced Stationary Syndrome Decoding (SSD) assumption [52, 2], (2) a new variant of SSD that we propose enables a more significant reuse of PIR queries, further improving concrete efficiency; (3) standard PIR optimization techniques, such as upload-download balancing [33], are used to further optimize the PIR cost. These optimizations are detailed in depth in Sections 5 and 6. See technical overview below for further discussion.

1.2 Technical Overview

1.2.1 The core framework.

The main existing approaches for concretely efficient maliciously secure computation either rely on the “SPDZ blueprint” which builds on authenticated multiplication triples (AMTs), or alternatively on the Fully Linear IOPs (FLIOPs) approach [14, 27, 28]. Both approaches involve significant sources of overhead, primarily in the multi-party setting or over small fields. A common technical idea in both approaches is to verify computation over some field $\mathbb{F}$ by computing a public random linear function of multiplication triples, possibly applying additional computation on top of it based on public values generated by the underlying semi-honest protocol. The linear function is chosen from some large space of challenges where each challenge r corresponds to a linear map $\text{Auth}_r : \mathbb{F}^{3n} \rightarrow \mathbb{F}^k$ for some small k and where $3n$ corresponds to the n triples.

An insecure way to compute Auth_r is by having each party locally compute Auth_r on its share of the triples and then sum the shares. Unfortunately, parties can trivially cheat as nothing binds them to provide the correct share. Such binding can be achieved by authenticating the triples, resulting in authenticated shares of the output $\text{Auth}_r(\vec{t})$, where $\vec{t}$ is the vector of triples additively shared between the parties.

Sparse-Tensored PCGs. We recall that concretely efficient approaches for generating several MTs (a_i, b_i, c_i) rely on PCGs from LPN-style assumptions. In particular, such constructions generate triples that have a *sparse representation* in the sense that $\vec{t} = G \cdot \vec{e}$ for some public linear mapping $G : \mathbb{F}^{3n \times N}$ and a *sparse* vector $\vec{e}$ of length $N > n$ and weight $w \ll N$. We call such PCGs *sparse-tensored* PCGs¹ (ST-PCGs) and provide a formal definition in Section 2.2. Existing state-of-the-art PCGs for MT (and AMT) from different flavors of LPN can all be cast in this framework [19, 12, 54, 52].

Reducing authentication to PIR. Our main observation is that when the MT vector $\vec{t}$ is generated from such an ST-PCG, then $\text{Auth}_r \cdot \vec{t} = \text{Auth}_r \cdot G \cdot \vec{e}$. Thus, rather than authenticate the $\vec{t}$ it suffices to authenticate the sparse vector $\vec{e}$ and then locally compute $\text{Auth}_r \cdot G \cdot \vec{e}$. Since $\vec{e}$ is sparse, authenticating it only requires holding authenticated shares of the w non-zero locations of $\vec{e}$ and their corresponding payloads. While these do not immediately introduce additive shares of $\vec{e}$, we show that such authentication suffices.

At this point comes the key observation of this work: we can view the matrix $\text{Auth}_r \cdot G$ as a (public) database with k rows and N columns, where the sparse vector $\vec{e}$ defines a linear combination of w secret rows in the database whose weights are defined by the nonzero payloads in $\vec{e}$. Moreover, in all existing ST-PCG constructions, a maliciously secure setup phase anyway needs to generate authenticated shares of the w non-zero locations of $\vec{e}$ and their corresponding payloads—exactly what is needed to compute $\text{Auth}_r \cdot G \cdot \vec{e}$.

This leads us to define a new variant of private information retrieval (PIR) that we call *authenticated sparse linear PIR* (ASL-PIR). ASL-PIR allows the parties to jointly compute an authenticated linear combination of w rows of a public database of N rows and k columns, where each party holds authenticated shares of the w indices and their corresponding payloads. This ASL-PIR functionality can be realized from any PIR protocol even by trivially applying generic secure computation to

¹The term “tensored” reflects the fact that one part of $\vec{e}$ is a tensor product of the other two; this feature is not important for the following discussion.

its query generation and response reconstruction phases, which allows us to leverage the extensive literature on efficient PIR protocols.

Our main contribution is an end-to-end maliciously secure, multi-party protocol in the dishonest majority setting that leverages ASL-PIR to verify the correctness of a triples-based semi-honest protocol based on the observations above. Since PIR can be realized with polylogarithmic communication from distributed point functions (DPF) in the 2-party case [23] or from single-server PIR [53], or alternatively unconditionally with square-root communication in the general m -party case [33], we can leverage this to obtain our main result from Theorem 1.1.

In the following, we give a complete informal description of our main protocol to provide some context and intuition about the challenges it must overcome.

Generating unauthenticated triples: The protocol begins with the parties generating n *unauthenticated* m -party MTs using an ST-PCG seed generation protocol. Implementations of this functionality in the literature, e.g., [19, 12, 54], all follow a similar blueprint that is crucial for our protocol’s efficiency. The triples are generated by a process that: (1) Samples sparse vectors $\vec{e}_A$ and $\vec{e}_B$. (2) Computes their “tensor”: $\vec{e}_C = U(\vec{e}_A, \vec{e}_B)$ using a bilinear-map U which ensures that $\vec{e}_C$ is also sparse. (3) The i -th triple is set as $(\vec{a}[i], \vec{b}[i], \vec{c}[i])$ where $\vec{a} = G_A \vec{e}_A$, $\vec{b} = G_B \vec{e}_B$ and $\vec{c} = G_C \vec{e}_C$ for publicly known compressive linear maps G_A, G_B, G_C .

Authenticating the triples: In addition to the unauthenticated triples, the protocol outputs m -party *authenticated shares* of the non-zero positions $\vec{I}_a$ (resp., $\vec{I}_b, \vec{I}_c$) and the corresponding values at these positions $\vec{V}_a$ (resp. $\vec{V}_b, \vec{V}_c$) in $\vec{e}_A$ (resp. $\vec{e}_B, \vec{e}_C$). We can use any standard method for generating authenticated shares, such as information-theoretic MACs. A key observation of this work is that although the triples are unauthenticated, the authenticated sharing of the non-zero positions of the sparse vectors and the values in these positions is a *succinct* authentication of the triples. In other words, the n triples can be authenticated with a tag that is much smaller than n . While this succinct authentication tag is not compatible with the SPDZ-style verification (where authentication is needed for each triple), we present an alternative lightweight verification method that exploits this tag efficiently, based on ASL-PIR.

Semi-honest execution: At this point we already have all the correlated randomness needed for a semi-honest evaluation of C . The parties execute Beaver’s standard MT-based protocol for evaluating the circuit C [3], which is known to be secure “up to additive attacks” against malicious parties [43]. Since cheating may have taken place, the parties still cannot open the circuit’s output wires (which may leak extra information). Instead, for each wire w that is either an output wire or an input wire to a multiplication gate, all parties know a *masked* version $\hat{v}_w$ of the wire’s value v_w , such that $\hat{v}_w = v_w + r_w$ for some additively secret-shared mask r_w . This share need not be authenticated, in contrast to SPDZ-like protocols.

Verifying the semi-honest protocol: To verify that the semi-honest protocol was executed correctly, the parties need to check for each wire in the circuit that is an input to a multiplication gate or is a circuit output wire, that the value on the wire is correct given the public value on wires that feeds this wire. To this end, we begin with the observation that this check is equivalent to checking that an inner product between two vectors $\vec{a}, \vec{b}$ is equal to a public value. In this inner product, $\vec{a}$ corresponds to public values known to the parties, whereas $\vec{b}$ is the random payload

vector shared across the parties. Hence, the parties can compute an additive sharing of the inner product. However, opening an additive sharing towards checking its equality to 0 is not secure, since the parties can cheat. To resolve this without authenticating each of the multiplication triple payloads, we use the observation above: we can write $\langle \vec{a}, \vec{b} \rangle$ as $\langle \vec{a}, G \cdot \vec{e} \rangle$, where G is a public matrix and $\vec{e}$ is a secret sparse vector. The parties can thus compute a public database where each entry is $\langle \vec{a}, G \cdot \text{unit}(i) \rangle$, where $\text{unit}(i)$ is the unit vector whose i th entry is 1. To complete the verification, the parties need to compute a sparse linear combination of entries in the database, corresponding to the non-zero positions in $\vec{e}$ and their payloads. This can be done using the ASL-PIR functionality! In particular, the parties provide as input to the functionality the public database and their succinct authentication of $\vec{e}$, i.e., the authenticated shares of the non-zero positions $\vec{I}_a, \vec{I}_b, \vec{I}_c$, and the authenticated shares of the corresponding payloads $\vec{V}_a, \vec{V}_b, \vec{V}_c$.

Reduction to PIR: We encapsulate the functionality of ASL-PIR in $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ and provide multiple simple constructions for realizing it based on single- and multi-server PIR protocols from the literature; see Section 5. ASL-PIR can be reduced to standard PIR with a low overhead, implying that our approach can benefit from any future improvements in PIR protocols.

1.2.2 Optimizations

We introduce multiple optimizations to minimize the costs of realizing ASL-PIR, especially in the context of our verification protocol.

Leveraging PIR balancing techniques. Since our ASL-PIR functionality can be realized from any PIR protocol, it can also benefit from existing PIR optimizations. We exemplify the power of this reduction to PIR by applying PIR balancing - a very simple PIR optimization. This optimization allows us to balance the costs of PIR query generation and response reconstruction. This yields significant savings that we concretely measure in Section 7.

Amortizing costs via Stationary Syndrome Decoding. We further optimize our verification protocol by relying on Stationary Syndrome Decoding (SSD) [52], which allows us to split the computation in a large number q of “batches.” The triples in each batch are generated from a separate ST-PCG seed, while all q seeds share the same non-zero positions. In the context of ASL-PIR, this means that the query indices remain the same across a large number of PIR queries, allowing us to amortize the PIR query generation costs by an additional factor of q , on top of the PIR balancing optimization.

Doubly Stationary Syndrome Decoding. While SSD allows us to amortize the PIR query generation costs, the database generation and query response costs remain high. To address this, we further analyze the SSD assumption and introduce a slightly stronger variant called Doubly-SSD (DSSD), which allows us to reuse the ST-PCG matrix G as well! From our analysis and based on known attacks [52], we find that while the matrix G can be reused a limited number of times (bounded by the LPN noise weight), this number can be sufficiently large to significantly reduce the database generation and query response costs as we demonstrate by benchmarks in Section 7.

A second verification protocol based on ASL-PIR and zk-FLIOP. As explained above, the DSSD assumption essentially allows us to reuse the public database. Unfortunately, with the verification protocol described above, this is not possible, since the public database is computed as a function not only of G , but also the masked values on the circuit wires $\vec{a}$. As these are different in each batch, it is not clear how to leverage the DSSD assumption.

To this end, we present a second variant of our verification protocol, which follows the footsteps of [27, 28] and uses zero-knowledge FLIOP to verify the correctness of the inner product equation. In this solution, the parties are required to jointly compute linear query responses over the vector of multiplication triple payloads. Unlike [27], which used generic MPC with linear communication complexity to compute these, and [28], which used additively homomorphic encryption, we use our ASL-PIR machinery. The difference here, that enables leveraging the DSSD assumption, is that the public database is computed as a function of G and the FLIOP random queries, which can remain the same across different batches!

As another contribution of independent interest, we show for the first time how FLIOP-based verification can be carried out using *multi-round* FLIOP in the *distributed* prover approach. This enables us to significantly improve the verification overhead compared with prior works [27, 28] both in terms of communication and storage. Due to lack of space, this variant is presented in Appendix D (including background on FLIOPs).

Future directions. We believe that our reduction to PIR opens the door for many optimization approaches based on the growing literature on PIR. In particular, from our benchmarks it follows that amortizing the PIR query responses by means of PIR with preprocessing (see [34] and follow-up works) and batch PIR (see [49] and follow-up works) can lead to significant savings.

2 Preliminaries

Notation. We denote by λ and σ computational and statistical security parameters, respectively. We use $\mathbb{F}$ to denote a finite field, and $\hat{\mathbb{F}}$ to denote its extension field of size at least 2^σ . When $\mathbb{F}$ is large enough, then we take $\hat{\mathbb{F}} = \mathbb{F}$. m denotes the number of parties and P_i denotes the i -th party. We use the notation $[m]$ to denote the set $\{1, \dots, m\}$. We denote by $\llbracket v \rrbracket$ the additive sharing of v across m parties, and by $\llbracket v \rrbracket_i$ for $i \in [m]$ the additive share of the i -th party, with respect to some ordering of the parties. Similarly, we also denote by $\langle\langle v \rangle\rangle$ the *authenticated secret sharing* of value v as defined in Section 2.1. For a distribution X we denote by $x \in_R X$ the sampling of a value into variable x from distribution X . For a finite set S we denote by $x \in_R S$ the uniform sampling of a value from the set S into variable x . For a set $T \subseteq [\ell]$ we denote by $a[T]$ the set $\{a[i] \mid i \in T\}$. For a set $T \subseteq S$ we denote by $\bar{T}$ the complement $S \setminus T$.

Vectors and matrices Given two vectors $\vec{a}, \vec{b}$ we denote by $\vec{a} \parallel \vec{b}$ the *concatenation* of the vectors. For a vector $\vec{a}$ of length ℓ we denote by $\vec{a}[i]$ the i -th element in the vector. For $i \in \mathbb{Z}_n$ and field $\mathbb{F}$ we use $\text{unit}(i, n)$ (or $\text{unit}(i)$ when n is clear from the context) to denote the vector $\vec{y} \in \mathbb{F}^n$ such that $\vec{y}[i] = 1_{\mathbb{F}}$ and $\vec{y}[j] = 0$ for $j \neq i$. For a vector $\vec{v} \in \mathbb{F}^N$ we denote by $\text{weight}(\vec{v})$ the Hamming weight of $\vec{v}$, i.e. the number of non-zero entries in $\vec{v}$. Formally $\text{weight}(\vec{v}) = |\{i \in [N] \mid \vec{v}[i] \neq 0_{\mathbb{F}}\}|$. We say that $\vec{v} \in \mathbb{F}^N$ is w -sparse if $\text{weight}(\vec{v}) \leq w$.

Matrices are notated using upper case letters. To denote the i th row of a matrix A , we use the notation $\vec{A}[i, \cdot]$. Similarly, the j th column is $\vec{A}[\cdot, j]$.

Conventions. A protocol’s communication complexity is measured as the sum of the lengths of the messages sent by all parties together along the protocol. In the case of a protocol’s computational complexity, we consider the computational cost *per party*.

MPC Security Definition In this work we consider the problem of secure multi-party computation (MPC) in the presence of malicious adversaries who may arbitrarily deviate from the protocol. We follow the standard security definition of real/ideal worlds by [45, 5] given in Appendix A for completeness.

2.1 Secret Sharing Schemes

We use standard secret sharing schemes (**Share**, **Rec**) which unless stated otherwise are just additive m -party shares. We also rely on an extension of secret sharing called “authenticated secret sharing,” which provides security against malformed shares being given to the **Rec** method, preventing an adversary from forcing **Rec** to output a value that was not originally shared via **Share**. In this work we use a folklore statistically secure authenticated secret sharing scheme ($\text{Share}_{\text{Auth}}$, Rec_{Auth}) for sharing elements from a field $\mathbb{F}$. When the field $\mathbb{F}$ is not large enough, we consider an extension field $\hat{\mathbb{F}}$ of $\mathbb{F}$ of size at least 2^σ .

- $\text{Share}_{\text{Auth}}(v, \Delta)$ for $v \in \mathbb{F}$ outputs m -party additive shares of Δ , $\Delta \cdot v$ and v . We omit Δ from notations when clear from the context.
- $\text{Rec}_{\text{Auth}}(\llbracket v \rrbracket)$ reconstructs v , and then checks whether $\Delta^* \cdot v = (\Delta \cdot v)^*$. If the equality holds, the parties output v and otherwise the output is $\perp$.

It can be shown that the success probability of violating the authentication property is negligible in σ . Thus, in the rest of this work $\llbracket v \rrbracket$ is the shorthand for $\llbracket v, \Delta v \rrbracket$ where Δ is an authentication key shared between the parties.

For completeness we give a formal definition of secret sharing and authenticated secret sharing in Appendix B.

Observe that this secret sharing scheme is *linear*, meaning that linear operations over the secret can be computed locally by the parties without any interaction. Hence, we can define addition and multiplication-by-a-scalar operations, such that for any $\llbracket u \rrbracket, \llbracket v \rrbracket$ and a public scalar α , it holds that $\llbracket u \rrbracket + \llbracket v \rrbracket = \llbracket u + v \rrbracket$, $\alpha \cdot \llbracket u \rrbracket = \llbracket \alpha \cdot u \rrbracket$ and $\alpha + \llbracket u \rrbracket = \llbracket \alpha + u \rrbracket$.

2.2 Sparse-Tensored PCG

Our results leverage the common structure of a collection of constructions of pseudorandom correlation generators (PCGs) for (unauthenticated) multiplication triples from LPN and other LPN-like assumptions [18, 20, 13, 54]. While these constructions differ in some of their internal details, they all share a common high-level structure which we encapsulate in the following definition under the collective term — *sparse-tensored PCGs*, or ST-PCGs for short.

Definition 2.1 (Sparse-tensored PCG for triples). *A sparse-tensored-PCG (ST-PCG) for generating $n = n(\lambda)$ multiplication triples over a field $\mathbb{F}$ is a PCG (**Gen**, **Expand**) parameterized by:*

- *Sparsity parameter $w(\lambda)$ and intermediate parameters $N' > N > n$, all polynomial in the security parameter λ (examples below);*

- Reusable, public compressive linear maps $G_a = G_{a,\lambda}, G_b = G_{b,\lambda}$ mapping from $\mathbb{F}^N \rightarrow \mathbb{F}^n$; we let the compression factor $R = N/n$.
- Compressive linear map $G_c = G_{c,\lambda}$ mapping $\mathbb{F}^{N'} \rightarrow \mathbb{F}^n$; and
- Bilinear map $U = U_\lambda : \mathbb{F}^N \times \mathbb{F}^N \rightarrow \mathbb{F}^{N'}$.

The ST-PCG algorithms have the following structure:

- **Gen**: Samples random w -sparse vectors $\vec{e}_a^i, \vec{e}_b^i$ in $\mathbb{F}^N$ for each party P_i . Sets $\vec{e}_a = \sum \vec{e}_a^i$ and $\vec{e}_b = \sum \vec{e}_b^i$, and computes a $(mw)^2$ -sparse vector $\vec{e}_c = U(\vec{e}_a, \vec{e}_b) \in \mathbb{F}^{N'}$. The PCG seed is given by $\vec{e}_a^i, \vec{e}_b^i, \vec{e}_c^i$ where $\vec{e}_c^i$ is an m -party (compressed) additive share of $\vec{e}_c$.

In addition to the m PCG seeds, we assume that **Gen** additionally outputs authenticated shares of ordered pairs of non-zero coordinates of the sparse vectors $E_\alpha = \{(i, e_\alpha[i]) : e_\alpha[i] \neq 0\}$, for each $\alpha \in \{a, b, c\}$. (In particular, $|E_a|, |E_b| \leq mw$ and $|E_c| \leq (mw)^2$.) A formal description of the functionality for the authenticated variant of **Gen** is given in $\mathcal{F}_{\text{Triples}}$.

- **Expand**: Each party computes its share of $\vec{a} = G_a(\vec{e}_a) \in \mathbb{F}^n, \vec{b} = G_b(\vec{e}_b) \in \mathbb{F}^n$, and $\vec{c} = G_c(\vec{e}_c) \in \mathbb{F}^n$ as its share of the n multiplication triples $(\vec{a}, \vec{b}, \vec{c})$.

At a high level, in the LPN-like based constructions of ST-PCGs, the $\vec{e}_a, \vec{e}_b$ correspond to sparse LPN (or Ring-LPN/QA-SD) noise vectors, so that the corresponding vectors $\vec{a}, \vec{b}$ are pseudorandom. The longer vector $\vec{e}_c = U(\vec{e}_a, \vec{e}_b) \in \mathbb{F}^{N'}$ corresponds to some degree-2 function on $\vec{e}_a$ and $\vec{e}_b$ for which the compressed version $G_c(\vec{e}_c)$ will correctly encode the pointwise products of the vectors $\vec{a}$ and $\vec{b}$.

Theorem 2.1 (ST-PCG constructions [18, 19, 13, 54]).

- Based on the LPN assumption with constant rate $1 - 1/R$ over $\mathbb{F}$ and noise $w = w(\lambda)$, there exists m -party ST-PCG for n unauthenticated multiplication triples where **Gen** has communication complexity $O(m^2 w^2 \text{polylog}(n))$ and **Expand** has computational complexity $O(m^2 w^2 n^2)$.
- Under the Ring-LPN variant of the previous assumption (from [19], Definitions 3.1 and 3.2), or under the Search-QA-SD assumption (from [13], Definition 16) there exists m -party ST-PCG for multiplication triples where **Gen** has communication complexity $O(m^2 w^2 \text{polylog}(n))$ and **Expand** has computational complexity $\tilde{O}(R^2 n) + O(mw^2 n)$ (per party).

Moreover, the linear compressing map C of the PCG is computable by an arithmetic circuit (containing only addition and scalar multiplication gates) of quasilinear size $\tilde{O}(n)$.

We remark that the specific computational complexity statement in the second bullet above is important as it will enable us to apply the transposition principle to reason about the computational cost of evaluating the *transpose* map of the corresponding linear function C . Indeed, for this goal it would not suffice to instead require C to have quasilinear computation in the RAM model or to be computable using circuits that additionally involve multiplications.

2.3 Syndrome Decoding and its Variants

The problem of Syndrome Decoding (SD) arises naturally in linear error correcting codes. Consider a linear code C over a field $\mathbb{F}$, i.e. $C \subseteq \mathbb{F}^k$ for some k , and its parity check matrix $\mathbf{G} \in \mathbb{F}^{n \times k}$. Then,

for every code word $\mathbf{x} \in C$, we have $\mathbf{G}\mathbf{x} = \mathbf{0}$. To decode a received message $\mathbf{y} = \mathbf{x} + \mathbf{e}$, where $\mathbf{e}$ is a sparse error vector, it is sufficient to decode the *syndrome* $\mathbf{G}\mathbf{e}$, i.e. map $\mathbf{G}\mathbf{e}$ to the error $\mathbf{e}$, exploiting the fact that $\mathbf{G}\mathbf{y} = \mathbf{G}(\mathbf{x} + \mathbf{e}) = \mathbf{G}\mathbf{e}$.

This decoding technique is efficient for some code families, e.g. Reed-Solomon codes, but for general linear codes it is thought to be hard. This computational hardness is a disadvantage for coding theory, but is very useful for cryptography, e.g. in the construction of PCGs [17, 19, 54, 52]. The formal definition of the hardness assumption follows.

Definition 2.2 (Syndrome Decoding). *Syndrome Decoding (SD) is parameterized by an implicit computational security parameter λ , a field $\mathbb{F}$, dimensions $n, k \in \mathbb{N}$ with $n < k$, and distributions $\mathcal{L}$ over $\mathbb{F}^k$ and $\mathcal{G}$ over $\mathbb{F}^{n \times k}$. For samples $\mathbf{e} \in_R \mathcal{L}$ and $\mathbf{G} \in_R \mathcal{G}$ the $(n, k, \mathbb{F}, \mathcal{L}, \mathcal{G})$ -SD assumption states that*

$$\{(\mathbf{G}, \mathbf{G}\mathbf{e})\} \approx \{(\mathbf{G}, \mathbf{x}) \mid \mathbf{x} \in_R \mathbb{F}^k\},$$

where $\approx$ denotes computational indistinguishability.

Many PCGs described in the cryptographic literature require two components: the SD assumption and a scheme for Distributed Point Functions (DPF) or Punctured Pseudorandom Functions (PPRF). The DPF or PPRF is used to share a sparse vector $\mathbf{e}$ between the participants in the PCG protocol. This vector is later used in conjunction with a public matrix $\mathbf{G}$ to generate the desired correlation, relying on the SD assumption for security.

Motivated by the goal of improving the efficiency of PCG constructions by reducing the number of independent DPF or PPRF invocations, [52] introduced the *Stationary Syndrome Decoding* assumption (SSD). This assumption considers q vectors that are generated by the product of q public matrices and q sparse vectors over a field $\mathbb{F}$ such that the non-zero locations of all vectors are *identical*, while the value in each such location is sampled randomly from $\mathbb{F}$ (including 0). The formal definition of the SSD assumption uses $\odot$ to denote pointwise product of two vectors over $\mathbb{F}$.

Definition 2.3 (Stationary Syndrome Decoding (SSD) [52]). *Stationary Syndrome Decoding (SSD) is parameterized by an implicit computational security parameter λ , a field $\mathbb{F}$, dimensions $n, k, q \in \mathbb{N}$ with $n < k$, and distributions $\mathcal{L}$ over $\mathbb{F}^k$ and $\mathcal{G}$ over $\mathbb{F}^{n \times k}$. Sample $L \in_R \mathcal{L}$, random and independent $V_1, \dots, V_q \in \mathbb{F}^k$ and random and independent $\mathbf{G}_1, \dots, \mathbf{G}_q \in_R \mathcal{G}$ and compute $\mathbf{e}_i = L \odot V_i, i = 1, \dots, q$. The $(n, k, q, \mathbb{F}, \mathcal{L}, \mathcal{G})$ -SSD assumption states that*

$$\{(\mathbf{G}_i, \mathbf{G}_i \mathbf{e}_i)\}_{i \in [q]} \approx \{(\mathbf{G}_i, \mathbf{x}_i) \mid \mathbf{x}_i \in_R \mathbb{F}^n\}_{i \in [q]}.$$

The security analysis of [52] shows that the SSD and the SD assumptions have similar resistance to most known attacks for a wide range of parameters and for many distributions $\mathcal{G}$ over $\mathbb{F}^{n \times k}$. An exception to this rule is when the support of $\mathcal{G}$ is a singleton $\{\mathbf{G}\}$, $\mathbf{G} \in \mathbb{F}^{n \times k}$. In this case, $\mathbf{G} = \mathbf{G}_i$ for all $i \in [q]$. As [52] observes, if the sparsity t of the vectors $\mathbf{e}_i, i \in [q]$ is smaller than the number of vectors q , then $\mathbf{G}\mathbf{e}_1, \dots, \mathbf{G}\mathbf{e}_q$ are linearly dependent and therefore easily distinguishable from q random matrices and q random vectors.

The case of SSD using the same matrix $\mathbf{G}$ has efficiency benefits for our construction, motivating the question of whether there is some number of iterations for which it is still secure. We study this question in the framework of a variant of SSD, which we call *Doubly Stationary Syndrome Decoding* (DSSD). In SSD there are q instances of SD, with the same error locations and freshly sampled matrices in each instance. In DSSD there are again q instances of SD, but once a matrix is sampled, it is used r times, and therefore there are q/r fresh matrices that are sampled to compute the q

output vectors. Note that if $r = 1$ then DSSD is identical to general SSD, while if $r = q$ then DSSD is identical to the SSD variant described previously with a singleton support for the distribution $\mathcal{G}$. Definition 2.4 formally defines the DSSD hardness assumption.

Definition 2.4 (Doubly Stationary Syndrome Decoding). *Doubly Stationary Syndrome Decoding (DSSD) is parameterized by an implicit computational security parameter λ , a field $\mathbb{F}$, dimensions $n, k, q, r \in \mathbb{N}$ with $n < k$, $r \leq q$, and distributions $\mathcal{L}$ over $\mathbb{F}^k$ and $\mathcal{G}$ over $\mathbb{F}^{n \times k}$. Sample $L \in_R \mathcal{L}$, random and independent $V_1, \dots, V_q \in \mathbb{F}^k$ and random and independent $\mathbf{G}_1, \dots, \mathbf{G}_{\lceil q/r \rceil} \in_R \mathcal{G}$ and compute $\mathbf{e}_i = L \odot V_i$. The $(n, k, q, r, \mathbb{F}, \mathcal{L}, \mathcal{G})$ -DSSD assumption states that*

$$\left\{ (\mathbf{G}_j, \mathbf{G}_j \mathbf{e}_i) \mid j = \left\lfloor \frac{i}{r} \right\rfloor + 1 \right\}_{i \in [q]} \approx \left\{ (\mathbf{G}_j, \mathbf{x}_i) \mid j = \left\lfloor \frac{i}{r} \right\rfloor + 1, \mathbf{x}_i \in_R \mathbb{F}^n \right\}_{i \in [q]}.$$

We argue that DSSD is secure with careful choice of parameters by analyzing the effectiveness of several classes of cryptanalytic attacks on DSSD. The attacks we consider include the previous rank attack that exploits overuse of the same matrix, attacks in the linear test framework of [36] including such classic attacks as Information Set Decoding [56, 58] or BKW [10], and algebraic attacks including [29]. The security analysis appears in Appendix C.

3 Ideal Functionalities

In this section, we define several functionalities that the parties access in our protocol. We begin with several standard functionalities described below:

- $\mathcal{F}_{\text{Mul}}$: this functionality receives authenticated shares of two field elements $u, v \in \mathbb{F}$ from the parties and returns authenticated shares of their product $w = u \cdot v$ with malicious security.
- $\mathcal{F}_{\text{Coin}}$: this functionality is parameterized by a field $\mathbb{F}$. Upon request, it hands the parties fresh random elements in the field. When the parties need many random elements at once, they can use the popular optimization of asking $\mathcal{F}_{\text{Coin}}$ to deliver a single element that is used as a seed to a PRG, from which the randomness is derived.
- $\mathcal{F}_{\text{Rand}}$: parameterized by a field $\mathbb{F}$, upon request, this functionality chooses a random field element v and hands the parties an authenticated secret sharing $\langle\langle v \rangle\rangle$. Note that in most implementations of this functionality in the literature, the adversary is allowed to choose the corrupted parties' shares.

We stress that in our constructions the above functionalities are called a number of times that is *independent* of the computed circuit size. Thus, any secure implementation will suffice.

3.1 Sparse Vector Generation Functionality

Many of the functionalities and protocols we present in this work rely on a basic building block that we denote by $\mathcal{F}_{\text{DMPF}}$ – a corruptible functionality that allows a set of parties to generate additive shares of a sparse vector from authenticated shares of the non-zero indices and their corresponding payloads. Only the non-zero positions of the vector are authenticated. The functionality $\mathcal{F}_{\text{DMPF}}$ (abbr. distributed multi-point function) is defined in Functionality 3.2. We also denote by $\mathcal{F}_{\text{UV}}$ a special case of $\mathcal{F}_{\text{DMPF}}$ where $w = 1$. The algorithm is given in Algorithm 3.1.

Algorithm 3.1 (DMPF). *A Sparse Vector Generating Algorithm*

Parameters: Output size n , weight w , group $\mathbb{G}$.

Input: Indices $\alpha_1, \dots, \alpha_w \in [n]$ and payloads $\beta_1, \dots, \beta_w \in \mathbb{G}$.

1. output $\sum_{i=1}^w \beta_i \cdot \text{unit}(\alpha_i, n)$
-

Functionality 3.2 ($\mathcal{F}_{\text{DMPF}}$). *A functionality for generating shares of a sparse vector*

$\mathcal{F}_{\text{DMPF}}$ is parameterized by a group $\mathbb{G}$, a vector length $n = p^k$, m parties and weight $w \in \mathbb{N}$.

1. $\mathcal{F}_{\text{DMPF}}$ receives authenticated shares of indices $\langle\alpha_1\rangle, \dots, \langle\alpha_w\rangle \in \mathbb{F}_p^k$ and payloads $\beta_1, \dots, \beta_w \in \mathbb{G}$.
 2. If a set of parties S are corrupted, $\mathcal{F}_{\text{DMPF}}$ waits to receive $\vec{v}_i \in \mathbb{G}^n$ from each $i \in S$.
 3. Let $\vec{v} = \text{DMPF}(\alpha_1, \dots, \alpha_w, \beta_1, \dots, \beta_w)$.
 4. For each $i \in [m] \setminus S$ $\mathcal{F}_{\text{DMPF}}$ samples a random vector $\vec{v}_i \in_R \mathbb{G}^n$ under the constraint that $\sum_{i=1}^m \vec{v}_i = \vec{v}$.
 5. $\mathcal{F}_{\text{DMPF}}$ sends each party P_i its output $\vec{v}_i$.
-

This functionality can be realized from maliciously secure setup protocols for *Distributed Point Functions* (DPFs) [23, 25] for unit vectors and summing up the resulting vectors. Such protocols exist (with a single bit of leakage) in [19, 54] for the two-party setting. Recent works have also considered direct setup protocols for efficient realizations of $\mathcal{F}_{\text{DMPF}}$ [52, 21, 2] based on DPFs combined with batch codes [49], OKVS [21], and other techniques, with efficiency summarized in the following theorem. In the 2-party setting, existing state-of-the-art maliciously secure DPF setup protocols [19, 54] can be used to realize $\mathcal{F}_{\text{UV}}$ with a single bit of leakage, communication growing logarithmically in the vector size n , and computation cost dominated by a linear number of PRG evaluations in n . Existing literature implies a feasible maliciously secure setup protocol with communication scaling sublinearly in n and computation $O(n + \text{poly}(t))$. Our construction will use such a protocol as a black-box and we refer the reader to the above works for details. Since the setup of $\mathcal{F}_{\text{DMPF}}$ will be amortized away in our final construction, we will not focus on optimizing this cost further in this work.

Functionality 3.3 ($\mathcal{F}_{\text{Triples}}$). *Generating multiplication triples from ST-PCGs*

Parameters: m parties $P_1, \dots, P_m$, field $\mathbb{F}$, triples count n , weight t , sparse vector sizes N, N' , bilinear mapping $U : \mathbb{F}^N \times \mathbb{F}^N \rightarrow \mathbb{F}^{N'}$ and compressive linear mappings $G_A, G_B \in \mathbb{F}^{n \times N}$ and $G_C \in \mathbb{F}^{n \times N'}$. The bilinear mapping U is assumed to be sparsity-preserving (i.e. $\text{weight}(U(x, y)) \leq \text{weight}(x) \cdot \text{weight}(y)$ for any $x, y \in \mathbb{F}^N$).

1. Sample indices $\vec{I}_\sigma \in [N]^{mt}$ and payloads $\vec{V}_\sigma \in \mathbb{F}^{mt}$ for $\sigma \in \{A, B\}$.
 2. Compute $\vec{e}_\sigma \leftarrow \text{DMPF}(\vec{I}_\sigma, \vec{V}_\sigma)$ for $\sigma \in \{A, B\}$.
 3. Compute $A \leftarrow G_A \vec{e}_A$, $B \leftarrow G_B \vec{e}_B$.
 4. Compute $\vec{e}_C \leftarrow U(\vec{e}_A, \vec{e}_B) \in \mathbb{F}^{N'}$ of weight at most $(mt)^2$ and set $C \leftarrow G_C \vec{e}_C$.
 5. Let $\vec{I}_C \in [N']^{(mt)^2}$ and $\vec{V}_C \in \mathbb{F}^{(mt)^2}$ be the non-zero indices and payloads in $\vec{e}_C$.
 6. For each corrupted P_σ , receive $A_\sigma, B_\sigma, C_\sigma \in \mathbb{F}^n$ from the adversary.
 7. For each non-corrupted P_σ sample C_σ randomly such that $\sum_{\sigma \in [m]} C_\sigma = C$.
 8. Send each party P_σ its output (A_i, B_i, C_i) and $\langle\langle \vec{I}_\sigma \rangle\rangle, \langle\langle \vec{V}_\sigma \rangle\rangle$ for $\sigma \in \{A, B, C\}$.
-

3.2 Sparse-Tensored PCG Functionality

The $\mathcal{F}_{\text{Triples}}$ corruptible functionality enables m parties to securely generate n instances of unauthenticated multiplication triples. We refer the reader to Section 2.2 for background on sparse-tensored PCGs. The functionality receives succinct authenticated shares of the vectors, where succinctness is achieved by providing only authenticated shares of the non-zero indices of the vectors ($\langle\langle I_\sigma \rangle\rangle$) and the values at these indices ($\langle\langle V_\sigma \rangle\rangle$) for $\sigma \in \{A, B\}$. We provide the formal functionality description in Functionality 3.3. Instantiations of this functionality can be obtained from existing constructions of sparse-tensored PCGs [18, 20, 13, 54], with all of them working in the $\mathcal{F}_{\text{DMPF}}$ -hybrid model. That is, they assume the existence of a protocol realizing $\mathcal{F}_{\text{DMPF}}$ and often provide one. We note that a maliciously secure setup protocol for $\mathcal{F}_{\text{DMPF}}$ in some of these works builds on a *leaky* variant of our functionality and shows that, with a small change in the underlying LPN assumption, this leakage can be eliminated. Since the soundness of our authentication protocol does not rely on the secrecy of the generated triples (but rather on the secrecy of the PIR query response's authentication key), we abstract away the details of the leakage to keep our presentation simple.

3.3 Authenticated Sparse Linear PIR Functionality

Traditionally, Private Information Retrieval (PIR) allows a client to privately query a public database held by one or more servers [33, 53].

Our main protocol relies on an extended notion of PIR encapsulated by $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$, a functionality

Functionality 3.4 ($\mathcal{F}_{\text{PIR}}^{\text{ASL}}$). *Authenticated Sparse Linear PIR*

Parameters: Field $\mathbb{F}$, query and database batching parameters q , database $DB \in \mathbb{F}^{M \times q}$ of size $M = p^k$ with each entry being a size- q vector over $\mathbb{F}$, number of parties m , query weight w .

1. Receive w database indices to be queried $\langle\langle \alpha_1 \rangle\rangle, \dots, \langle\langle \alpha_w \rangle\rangle \in \mathbb{F}_p^k$ and for each query index a corresponding weight $\langle\langle \beta_1 \rangle\rangle, \dots, \langle\langle \beta_w \rangle\rangle \in \mathbb{F}^q$, all shared under the same authentication key $\Delta \in \hat{\mathbb{F}}$.
 2. Output authenticated sharing of the size- q vector $\langle\langle \sum_{i=1}^w \beta_i \odot DB_{\alpha_i} \rangle\rangle_{\mathbb{F}}$.
 Comment: $\odot$ denotes the point-wise product of two vectors
-

that allows a set of parties to privately query a public database where the set of possible queries are linear combinations of up to t database entries, and the linear combination coefficients should remain private as well. We parameterize the functionality by a query batching parameter $q \in \mathbb{N}$. This will later allow us to query q different linear combinations of the same set of indices in q (possibly different) databases. We denote by $\hat{\mathbb{F}}$ an extension of $\mathbb{F}$ with at least 2^σ elements. We provide the formal functionality description in Functionality 3.4. Constructions of ASL-PIR can be realized directly from standard (batch) PIR for the same privacy threshold, with communication scaling with the set size $w = |S|$ and a statistical authenticator of size σ . This can be improved for several PIR constructions where the server-to-client message(s) have a natural linear structure, in which case the server's message to the client need not grow in size. We note that in fact we can realize $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ from any protocol that securely realizes a natural, maliciously secure PIR functionality, thus we can benefit from any improvements in the PIR literature. The following two constructions follow directly from the 2-server PIR based on Distributed Point Functions (DPF) implied by [25] and multi-server PIR based on multi-party DPFs implied by [24] with the PIR balancing technique from [32, 33].

Theorem 3.5 (ASL-PIR [24, 25, 32, 33]). *Denote by $M = M(\lambda)$ the number of database entries, and $s = s(\lambda)$ such that each database entry is an element of $\mathcal{R}^s$ for some ring $\mathcal{R} = \mathcal{R}(\lambda)$. Let σ be a statistical security parameter.*

- (2-party DPF) *Assuming one-way functions, there exists 2-server weight- w ASL-PIR with client-to-server communication $O(w(\lambda \log M + \sigma))$ bits and server-to-client communication $O(s \lceil \log |\mathcal{R}| \rceil + \sigma)$ bits.*
- (Multi-party DPF) *Assuming one-way functions, there exists m -server weight- w ASL-PIR with privacy against $m - 1$ servers with client-to-server communication $O(\sigma w s 2^m M^{1/3})$ and server-to-client communication $O(\sigma s 2^m M^{1/3})$.*
- (Single-server PIR) *Assuming single-server PIR with polylog communication, there exists m -server weight- w ASL-PIR with privacy against $m - 1$ servers with both client-to-server communication $w \cdot \text{polylog}(M)$ and server-to-client communication $\text{polylog}(M)$.*

4 Our MPC Protocol

4.1 The Semi-Honest Protocol

Our semi-honest circuit evaluation protocol Π_{SH}^+ (where “+” refers to security against additive attacks) is a standard triples-based semi-honest protocol in which parties silently evaluate addition gates and communicate two field elements per multiplication gate. This protocol satisfies two important properties:

1. At the end of the protocol each party holds a *masked* value $\hat{v}_w$ for each wire w that is either an output wire or an input wire to a multiplication gate such that $\hat{v}_w = v_w + r_w$ for some mask r_w .
2. The protocol is secure-up-to-additive-attack [44]: If an adversary corrupts some parties and changes their inputs by some additive values, then the view of the adversary is indistinguishable from the view in an execution where the corrupted parties’ inputs are changed by these additive values and all parties follow the protocol honestly.

The protocol is fully described in [47], Protocol 12.

4.2 Verifying Π_{SH}^+ in the $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ -Hybrid Model

In this section we present Π_{Vrfy} , a protocol for verifying the correct execution of the semi-honest protocol Π_{SH}^+ . The protocol takes as input the *masked* values $\hat{v}_w$ for each wire w that is either an output wire of the circuit or an input wire to a multiplication gate, and a *succinct* representation of the masks — authenticated shares of the non-zero positions in $\vec{e}_a, \vec{e}_b, \vec{e}_c$ and their values in these non-zero positions.

The semi-honest protocol based on Beaver triples [3] has the property that for each circuit wire w that is an input to a multiplication gate or an output wire, the parties learn a masked value $\hat{x}_w = x_w - r_w$ and hold additive shares of the mask r_w (coming from the multiplication triples).

As in [27, 28], the parties verify the consistency of the values shared on each wire that is an input to a multiplication gate, with the values shared on the wires that precede it. Let W be the set of input wires to multiplication gates and output wires of the circuit. For each wire $w \in W$, the parties verify that they hold a sharing of the correct value on w , given the sharings they hold on wires that feed w . Specifically, let G_w be the set of multiplication gates that feed w (i.e., that between their output wire and w there are no other multiplication gates). For each $g \in G_w$, let x_1^g, x_2^g be the two input wires to g . The parties wish to verify that $x_w - \sum_{g \in G_w} x_1^g \cdot x_2^g = 0$. Instead, it suffices for them to verify that

$$\begin{aligned} p &= \sum_{w \in W} \alpha_w \cdot (x_w - \sum_{g \in G_w} x_1^g \cdot x_2^g) \\ &= \sum_{w \in W} \alpha_w \cdot \left(\hat{x}_w + r_w - \sum_{g \in G_w} (\hat{x}_1^g + r_1^g) \cdot (\hat{x}_2^g + r_2^g) \right) = 0 \end{aligned} \tag{1}$$

where the α_w are random elements jointly chosen by the parties.

Next, note that each gate g_ℓ can feed several wires. For each multiplication gate g_ℓ , let W^{g_ℓ} be the set of wires w for which $g_\ell \in G_w$ (i.e., that g_ℓ 's output feeds these wires). Then, let $\gamma_\ell = \sum_{w \in W^{g_\ell}} \alpha_w$.

Thus, Eq. (1) can be written as

$$\begin{aligned} p &= \sum_{w \in W} \alpha_w \cdot (\hat{x}_w + r_w) - \sum_{g_\ell \in \mathbf{mult}} \gamma_\ell \cdot ((\hat{x}_1^{g_\ell} + r_1^{g_\ell}) \cdot (\hat{x}_2^{g_\ell} + r_2^{g_\ell})) \\ &= \sum_{w \in W} \alpha_w \cdot \hat{x}_w - \sum_{g_\ell \in \mathbf{mult}} \gamma_\ell \cdot (\hat{x}_1^{g_\ell} \cdot \hat{x}_2^{g_\ell}) + \sum_{w \in W} \alpha_w \cdot r_w \\ &\quad - \sum_{g_\ell \in \mathbf{mult}} \gamma_\ell \cdot (\hat{x}_1^{g_\ell} \cdot r_2^{g_\ell} + \hat{x}_2^{g_\ell} \cdot r_1^{g_\ell}) - \sum_{g_\ell \in \mathbf{mult}} \gamma_\ell \cdot (r_1^{g_\ell} \cdot r_2^{g_\ell}). \end{aligned}$$

where **mult** is the set of multiplication gates. Now, setting

$$\Lambda = \sum_{w \in W} \alpha_w \cdot \hat{x}_w - \sum_{g_\ell \in \mathbf{mult}} \gamma_\ell \cdot (\hat{x}_1^{g_\ell} \cdot \hat{x}_2^{g_\ell}) \quad (2)$$

$$\Gamma = \sum_{w \in W} \alpha_w \cdot r_w - \sum_{g_\ell \in \mathbf{mult}} \gamma_\ell \cdot (\hat{x}_1^{g_\ell} \cdot r_2^{g_\ell} + \hat{x}_2^{g_\ell} \cdot r_1^{g_\ell} + (r_1 \cdot r_2)^{g_\ell}) \quad (3)$$

it follows that checking that $p = 0$ is equivalent to checking that $\Lambda + \Gamma = 0$. Next, observe that letting

$$\vec{a}' = ((\alpha_w)_{w \in W}, (-\gamma_\ell \cdot \hat{x}_1^{g_\ell}, -\gamma_\ell \cdot \hat{x}_2^{g_\ell}, -\gamma_\ell)_{g_\ell \in \mathbf{mult}}) \quad (4)$$

$$\vec{b}' = ((r_w)_{w \in W}, (r_2^{g_\ell}, r_1^{g_\ell}, (r_1 \cdot r_2)^{g_\ell})_{g_\ell \in \mathbf{mult}}) \quad (5)$$

we have that $\Gamma = \langle \vec{a}', \vec{b}' \rangle$ where $\vec{a}'$ is public and known to all parties, and $\vec{b}'$ is additively-shared across the parties. Observe that the first part $(r_w)_{w \in W}$ of the vector $\vec{b}'$ consists of copies of various elements from the triples $(r_2^{g_\ell}, r_1^{g_\ell}, (r_1 \cdot r_2)^{g_\ell})_{g_\ell \in \mathbf{mult}}$. Hence, this expression can be rearranged to modified vectors $\vec{a}, \vec{b}$ for which $\langle \vec{a}, \vec{b} \rangle = \langle \vec{a}', \vec{b}' \rangle$, and $\vec{b}$ is precisely the “payload” vector of the multiplication triples followed by the payloads of the circuit output wires (and in $\vec{a}$, the entry that will be multiplied with r_w consists of sum of the corresponding elements in $\vec{a}'$).

At this point, we depart from [27, 28], which relied on zk-FLIOP for degree-2 languages to prove the correctness of the inner product between $\vec{a}$ and $\vec{b}$. Instead, we exploit the sparse representation of the triples to replace zk-FLIOP by ASL-PIR. To this end, we observe that in this inner product, $\vec{a}$ is known and public at the verification step, and hence the inner product is in fact a linear function in $\vec{b}$, which is shared across the parties. Moreover, the vector $\vec{b}$ itself consists of the multiplication triples, which were computed as a linear mapping of a sparse vector. Specifically, a vector of triple payloads is computed as $G \cdot \vec{e}$, where G is known and $\vec{e}$ is a secret sparse vector. Let $\vec{e}_1, \dots, \vec{e}_{\text{weight}(\vec{e})}$ be the weight-1 vectors such that $\vec{e} = \sum_{j=1}^{\text{weight}(\vec{e})} \vec{e}_j$. Hence, we can write

$$\langle \vec{a}, \vec{b} \rangle = \langle \vec{a}, G \cdot \vec{e} \rangle = \langle \vec{a}, G \cdot (\sum_{j=1}^{\text{weight}(\vec{e})} \vec{e}_j) \rangle = \sum_{j=1}^{\text{weight}(\vec{e})} \langle \vec{a}, G \cdot \vec{e}_j \rangle$$

It follows that the parties can define a database of size N (the sparse vector size), where the i th entry is defined as $\langle \vec{a}, G \cdot \text{unit}(i) \rangle$. Then, to compute the desired $\langle \vec{a}, \vec{b} \rangle$, it suffices to take a sparse

linear combination of these entries, defined by the non-zero entries in $\vec{e}$. Given that the parties hold an authenticated secret sharing of the non-zero positions and their payloads, it only remains to send these to $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ and receive the result.

Our protocol is formally described in Protocol 4.1. Note that in the protocol there are three sparse vectors to consider ($\vec{e}_a, \vec{e}_b$ and $\vec{e}_c$). Thus, the size of the database is $2N + N'$, and when concatenating the three vectors that hold the non-zero positions, the parties need to shift the non-zero positions of the second and third vectors by N and $2N$, respectively (N is the length of the first and second vector). This can be done locally since the positions are shared via a linear secret sharing.

Remark 4.1 (The Complexity of Generating the Database). *Notice that the database $DB \in \hat{\mathcal{F}}^{2N+N'}$ satisfies $DB[i] = \langle \vec{a}_T, G \cdot \text{unit}(i) \rangle$. Thus $DB = G^T \cdot \vec{a}_T$. It follows from the transposition principle [7] that applying G and G^T have the same concrete efficiency up to a linear number of additions, thus generating the database is as efficient as applying the ST-PCG linear mapping G .*

4.3 The Main Protocol

Next, we present our multi-party protocol for the secure evaluation of arithmetic and boolean circuits in the $(\mathcal{F}_{\text{Triples}}, \mathcal{F}_{\text{PIR}}^{\text{ASL}}, \mathcal{F}_{\text{Rand}}, \mathcal{F}_{\text{Coin}})$ -hybrid model. The protocol is standard and works in a natural way. The parties call $\mathcal{F}_{\text{Triples}}$ to receive the PCG seeds and the authenticated secret sharing of the non-zero positions and their payloads. In addition, they call $\mathcal{F}_{\text{Rand}}$ to receive authenticated secret sharing of random masks for the circuit's input and output wires. Then, the parties compute the circuit by running the semi-honest multiplication protocol Π_{SH}^+ , while consuming a Beaver triple for each multiplication gate and random masks for input and output wire. Before revealing the outputs, the parties run the verification protocol Π_{Vrfy} . If the output of the protocol is **accept**, the parties proceed to reveal the outputs. It only remains to explain how to handle input and output wires. For each input wire, the parties reconstruct the mask to the input owner, who then broadcasts the masked input to the parties (this can be done with simple echo-broadcast). Similarly, for each output wire, the parties reconstruct the mask to the party who should receive the output. That party can then compute the output value by subtracting the mask. We prove the security of our protocol in Theorem 4.2. The proof can be found in Appendix F. Observe that the protocol has statistical security, since there may be an event where cheating in the semi-honest protocol took place, but the random linear combination in Eq. (1) yields 0.

Theorem 4.2. *Let f be a n -party functionality represented by an arithmetic circuit over a field $\mathbb{F}$. Then, our protocol securely computes f with statistical security and abort in the $(\mathcal{F}_{\text{Triples}}, \mathcal{F}_{\text{PIR}}^{\text{ASL}}, \mathcal{F}_{\text{Rand}}, \mathcal{F}_{\text{Coin}})$ -hybrid model, in the presence of malicious adversaries controlling any $\leq n - 1$ parties.*

5 Instantiating $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$

5.1 In the $\mathcal{F}_{\text{DMPF}}$ -hybrid model

In this section we provide a simple instantiation of $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ based on $\mathcal{F}_{\text{DMPF}}$. The protocol is silent in the $\mathcal{F}_{\text{DMPF}}$ -hybrid model and is given in Protocol 5.1.

Theorem 5.2 (Security of $\Pi_{\text{PIR}}^{\text{DMPF}}$). *Protocol 5.1 securely realizes $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ in the $\mathcal{F}_{\text{DMPF}}$ -hybrid model.*

Correctness follows from correctness of $\mathcal{F}_{\text{DMPF}}$, and security follows since the protocol is silent; the simulator can therefore play the role of $\mathcal{F}_{\text{DMPF}}$ and hand the adversary random output shares.

Theorem 5.3 (Efficiency of $\Pi_{\text{PIR}}^{\text{DMPF}}$). *Protocol 5.1 makes a single call to $\mathcal{F}_{\text{DMPF}}$ and has local computation cost of M operations in $\mathbb{F}$ and M in $\hat{\mathbb{F}}$.*

Remark 5.1. *We note that the indices sent to $\mathcal{F}_{\text{DMPF}}$ by $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ are the same indices sent to the DMPF algorithm by $\mathcal{F}_{\text{Triples}}$, suggesting that $\mathcal{F}_{\text{DMPF}}$ can be called once for both $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ and $\mathcal{F}_{\text{Triples}}$, thus halving the setup cost of our protocol.*

5.2 In the $\mathcal{F}_{\text{UV}}$ -hybrid model

We note two special cases where $\Pi_{\text{PIR}}^{\text{DMPF}}$ is suboptimal. The first is when the setup costs of $\mathcal{F}_{\text{DMPF}}$ are too high. The second is when we can benefit from reusing the same PIR query response multiple times. Regarding the latter, notice that the $\Pi_{\text{PIR}}^{\text{DMPF}}$ already outputs the queried linear combination of the database entries. When the same query *positions* are reused over the same database but the linear combination of them is different, it may be more efficient to query each position separately, obtain the responses and then apply a small-scale secure computation to obtain the desired linear combinations. Our protocol $\Pi_{\text{PIR}}^{\text{UV}}$ with batching parameter q and query weight w simply calls $\mathcal{F}_{\text{UV}}$ once for each noisy position to obtain authenticated shares of queried database entries. This costs wM (where M is the database size) operations in $\mathbb{F}$ and $\hat{\mathbb{F}}$. Next, the protocol computes q linear combinations of the w outputs using wq calls to $\mathcal{F}_{\text{Mul}}$. The protocol is given in Appendix E. Correctness holds since all inputs are authenticated under the same authentication key. We denote this protocol by $\Pi_{\text{PIR}}^{\text{UV}}$ and provide its efficiency in the following theorem.

Theorem 5.4. *There exists a protocol $\Pi_{\text{PIR}}^{\text{UV}}$ that securely realizes $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ in the $\mathcal{F}_{\text{UV}}$ -hybrid model by making w calls to $\mathcal{F}_{\text{UV}}$ and $2w - 1$ operations in $\mathbb{F}^q$ and in $\hat{\mathbb{F}}$.*

5.3 Optimizing $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ from balanced PIR

We recall that $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ can benefit from any improvements in PIR. In particular, the *PIR balancing* technique of [32, 33] can be applied to any such instantiation. While this optimization cannot be directly applied to our $\Pi_{\text{PIR}}^{\text{DMPF}}$ instantiation, we can apply it to our simpler variant $\Pi_{\text{PIR}}^{\text{UV}}$ and leverage this balancing technique for each call to $\mathcal{F}_{\text{UV}}$. Then, each call to $\mathcal{F}_{\text{UV}}$ is replaced with two calls to $\mathcal{F}_{\text{UV}}$ on a vector of size M/b and of size b respectively, for a balancing parameter $b \in \mathbb{N}$, and $< 2(b+1)$ calls to $\mathcal{F}_{\text{Mul}}$. After obtaining the authenticated shares of the w queried entries, we make the remaining wq calls to $\mathcal{F}_{\text{Mul}}$ to obtain the final output of the q linear combinations.

Theorem 5.5 (Efficiency of Balanced $\Pi_{\text{PIR}}^{\text{UV}}$). *The balanced variant of $\Pi_{\text{PIR}}^{\text{UV}}$ with balancing parameter b and batching parameter q makes w calls to $\mathcal{F}_{\text{UV}}$ on vectors of size M/b , w calls to $\mathcal{F}_{\text{UV}}$ on vectors of size b , and $(b+q)w$ calls to $\mathcal{F}_{\text{Mul}}$, and has local computation cost of $2w(M+b)$ field operations in $\mathbb{F}^q$ and in $\hat{\mathbb{F}}$.*

6 Optimizing Π_{Vrfy} from Stationary Syndrome Decoding

In this section we present significant optimizations that reduce the cost of Π_{Vrfy} . To present them, it is instructive to consider a variant of Π_{Vrfy} , denoted by Π_{Vrfy}^q , that for a parameter $q \in \mathbb{N}$ executes q

independent, equally sized instances of the verification procedure in parallel while reusing the same α_w . Since we make q verifications but verify q fewer multiplications in each check, soundness is preserved. Notice that Π_{Vrfy}^1 is simply Π_{Vrfy} . Thus, we view the n triples as if they were generated by q independent ST-PCG instances with matrices $G_1, \dots, G_q \in \mathbb{F}^{3n/q \times (2N_q + N'_q)}$, where each instance generates n/q triples.² At this point the protocol works similarly to the original Π_{Vrfy} by making q calls to $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ on databases smaller by a factor of q which we summarize in the following theorem:

Theorem 6.1. *For $n, q \in \mathbb{N}$, assuming there exists an ST-PCG for n/q MTs over $\mathbb{F}$ with parameters N_q, N'_q and vector sparsity w , the protocol Π_{Vrfy}^q for verifying the semi-honest execution of n multiplication gates by Π_{SH}^+ over $\mathbb{F}$ makes q calls to $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ on databases of size $2N_q + N'_q$ and query sparsity w^2 and batching parameter 1 and database records in $\hat{\mathbb{F}}$.*

Stationary Syndrome Decoding By leveraging the recently introduced *Stationary Syndrome Decoding* (SSD) assumption [52] we can benefit from the reuse of the same noisy positions to reduce the q calls to $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ to a single call to $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ on a database of size $2N_q + N'_q$ but where each entry is a q -sized vector in $\hat{\mathbb{F}}$. Since q can become arbitrarily large without affecting the soundness error [52], we can amortize away costs associated with the setup around the w non-zero entries, which dominate the cost of $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$, especially in the 2-party $\Pi_{\text{PIR}}^{\text{DMPF}}$.

Thus, we describe $\Pi_{\text{Vrfy}}^{q, \text{SSD}}$, a protocol similar to Π_{Vrfy}^q but now assuming that the q ST-PCG instances share the same noisy positions. The protocol $\Pi_{\text{Vrfy}}^{q, \text{SSD}}$ works similarly to Π_{Vrfy}^q except for the database generation and the calls to $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ as we describe below. The main idea is to exploit the repeated noisy positions to generate a single “batched” query to $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ with batching parameter q and the same sparsity w^2 . Thus, the database DB is generated by considering $DB_i \in \hat{\mathbb{F}}^{(2N_q + N'_q)}$ for $i \in [q]$ as in Π_{Vrfy}^q from the ST-PCG matrix G_i ; each has only $2N_q + N'_q$ rows and is built from the subset of elements in $\vec{a}_T$ that are associated with the i -th “batch”. Next, we stack all DB_i ’s next to one another as columns of the database, effectively forming a database in which each entry contains a vector in $\hat{\mathbb{F}}^q$. $DB \in \hat{\mathbb{F}}^{(2N_q + N'_q) \times q}$ with $DB[j] = (DB_1[j], DB_2[j], \dots, DB_q[j])$ for $j \in [2N_q + N'_q]$. Finally, the response from $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ is a vector of size q where each entry corresponds to the response of one of the q instances of Π_{Vrfy}^q . We summarize the above in Theorem 6.2.

Theorem 6.2. *For $n, q \in \mathbb{N}$, assuming $(n, k, q, \mathbb{F}, \mathcal{L}, \mathcal{G})$ -SSD and the existence of an ST-PCG for n/q MTs over $\mathbb{F}$ with parameters N_q, N'_q and vector sparsity w , the protocol $\Pi_{\text{Vrfy}}^{q, \text{SSD}}$ for verifying the semi-honest execution of n multiplication gates by Π_{SH}^+ over $\mathbb{F}$ makes a single call to $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ on databases of size $2N_q + N'_q$, query sparsity w^2 , query batching parameter q with database records in $\hat{\mathbb{F}}^q$.*

Concretely, the single call to $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ can reduce setup costs when realizing this functionality. For example, when realizing it from $\mathcal{F}_{\text{DMPF}}$ (in our protocol $\Pi_{\text{PIR}}^{\text{DMPF}}$), we need to generate only a single DMPF of the same weight w , with payloads larger by a factor of q .

Doubly-Stationary Syndrome Decoding At this point another significant bottleneck in the protocol of $\Pi_{\text{Vrfy}}^{q, \text{SSD}}$ is the database generation time which is still of size n and does not benefit from the batching parameter q . We introduce another optimization based on the Doubly-Stationary

²Notice that N_q and N'_q are smaller than N and N' . Concretely, we can consider $N'_q = N'/q$ and $N_q = N/q$.

Syndrome Decoding (DSSD) assumption from Definition 2.4 that allows us to reduce the database generation time by a factor of $\approx r$ under the assumption that $(n, k, q, r, \mathbb{F}, \mathcal{L}, \mathcal{G})$ -DSSD holds for appropriate parameters. Using existing approaches, reducing the database size seems impossible. Informally, the database construction depends on the size $3n$ vector $\vec{a}_T$ and it seems that a lower bound of $\Omega(n)$ is inherent for building the database. To eliminate the dependence on $\vec{a}_T$ we adopt a completely different verification approach based on Fully-Linear IOPs (FLIOPs) [14] which at a high level allows the parties to succinctly “authenticate” the triples generated by the ST-PCG *independently* of the concrete values in $\vec{a}_T$. The core of the verification protocol is that the parties jointly sample a random challenge r which defines an authentication matrix A_r . For verification, parties wish to learn the function $A_r \cdot \vec{T}$ where $\vec{T}$ is the vector of triples. Since the triples are generated by $\vec{T} = G \cdot \vec{e}$ from an ST-PCG matrix G and a sparse vector $\vec{e}$, the parties wish to learn $A_r \cdot G \cdot \vec{e}$. Thus by defining the database to be $DB = A_r \cdot G$ the parties need to learn the linear combination of a small number of elements from the database. We note that the database is exactly the same size as in previous approaches, and it has the same generation cost. Since we use FLIOPs in a black-box way, we defer presentation of $\Pi_{\text{Vrfy}}^{q, \text{DSSD}}$ with the full details to Appendix D while summarizing the main benefits in Theorem 6.3. In particular, we draw the reader’s attention also to the following corollary regarding the costs of the verification protocol when realizing $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ with $\Pi_{\text{PIR}}^{\text{UV}}$.

Theorem 6.3. *For $n, q, r \in \mathbb{N}$, assuming $(n, k, q, r, \mathbb{F}, \mathcal{L}, \mathcal{G})$ -DSSD and the existence of an ST-PCG for n/q MTs over $\mathbb{F}$ with parameters N_q, N'_q and vector sparsity w , the protocol $\Pi_{\text{Vrfy}}^{q, \text{DSSD}}$ for verifying the semi-honest execution of n multiplication gates by Π_{SH}^+ over $\mathbb{F}$ makes a single call to $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ on databases of size $2N_q + N'_q$, query sparsity w^2 , query batching parameter q , with database records in $\hat{\mathbb{F}}^{q/r}$.*

Corollary 6.1. *The costs of $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ in $\Pi_{\text{Vrfy}}^{q, r, \text{DSSD}}$ when realized with $\Pi_{\text{PIR}}^{\text{UV}}$ are dominated by database generation and query answering steps and are $\times r$ lower than the costs of Π_{Vrfy}^q . In particular, the costs of each of the steps are as follows: When $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ is realized directly from $\mathcal{F}_{\text{UV}}$ (as in our protocol $\Pi_{\text{PIR}}^{\text{UV}}$) with balancing parameter b , the costs associated with realizing $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ are as follows:*

- **Database Generation:** *For each block of r batches the database is reused, reducing the database generation cost by a factor of r compared to Π_{Vrfy}^q .*
- **Query Generation:** *$\mathcal{F}_{\text{UV}}$ is called w times on vectors of size $(2N_q + N'_q)$. When regular-noise DSSD is assumed, the vectors are of size $(2N_q + N'_q)/bt$. The output of $\mathcal{F}_{\text{UV}}$ is reused across all q batches, so the cost can be amortized.*
- **Query Answering:** *Since both the database and queries are reused within the same block of r batches, the cost of answering the queries is reduced by a factor of r compared to Π_{Vrfy}^q .*
- **Reconstruction:** *The cost of reconstructing the query answers requires a total of $bw + rw$ calls to $\mathcal{F}_{\text{Mul}}$ per batch.*

Remark 6.1. *Our approach for query reconstruction via direct calls to $\mathcal{F}_{\text{Mul}}$ is suboptimal. We note that reconstruction multiplies different linear combinations of the same w elements from the database q times, effectively requiring a secure matrix-vector product functionality. We leave this optimization for future work.*

7 Concrete Efficiency

Table 1: Cost factors in the 2-party setting between PCG-SPDZ, our baseline protocol Π_{MPC} and our protocol with SSD and Doubly-SSD optimizations. For SPDZ building blocks costs are all the same, though concrete costs will differ in how these building blocks are implemented. In our protocols we denote in green, blue, purple and orange the costs of generating the database, generating the PIR queries, answering PIR queries and reconstructing the query output, respectively.

Protocol		Semi-Honest	Authentication	Verification
PCG-SPDZ			$4c^2tq\sigma \cdot \text{DMPF}(\frac{n}{tq}, t, \mathbb{F}_4)$ $+ q\sigma \cdot \text{Lin}(\frac{n}{q}, \mathbb{F}_4)$	
This work			$q\sigma \cdot \text{Lin}(\frac{n}{q}, \mathbb{F}_4)$ $+ 4(ct)^2q \cdot \text{DPF}(\frac{n}{tqb}, \mathbb{F}_{2^\sigma})$ $+ 2c^2tn \cdot \text{Mul}_{\mathbb{F}_{2^\sigma}}$ $+ (ct)^2qb \pi_{\text{Mul}}^{\mathbb{F}_{2^\sigma}} $	$2n \cdot \text{Mul}_{\mathbb{F}_{2^\sigma}}$
	+ SSD	$4c^2tq \cdot \text{DMPF}(\frac{n}{tq}, t, \mathbb{F}_4)$ $+ q \cdot \text{Lin}(\frac{n}{q}, \mathbb{F}_4)$	$q\sigma \cdot \text{Lin}(\frac{n}{q}, \mathbb{F}_4)$ $+ 4(ct)^2 \cdot \text{DPF}(\frac{n}{b^*tq}, \mathbb{F}_{2^\sigma})$ $+ 2c^2tn \cdot \text{Mul}_{\mathbb{F}_{2^\sigma}}$ $+ (ct)^2qb^* \pi_{\text{Mul}}^{\mathbb{F}_{2^\sigma}} $	
	+ DSSD		$\frac{1}{r} \cdot q\sigma \cdot \text{Lin}(\frac{n}{q}, \mathbb{F}_4)$ $+ 4(ct)^2 \cdot \text{DPF}(\frac{n}{b^*tq}, \mathbb{F}_{2^\sigma})$ $+ \frac{1}{r} \cdot 2c^2tn \cdot \text{Mul}_{\mathbb{F}_{2^\sigma}}$ $+ (ct)^2qb^* \pi_{\text{Mul}}^{\mathbb{F}_{2^\sigma}} $	$10n \cdot \text{Mul}_{\mathbb{F}_{2^\sigma}}$

We begin by comparing, in the 2-party setting, the end-to-end cost of secure evaluation of a Boolean circuit in our protocol with a variant of SPDZ based on PCG for AMTs that we call PCG-SPDZ. We focus on $\mathbb{F}_2$ to keep the analysis simple. We use the ST-PCG of [54] for generating (A)MTs over $\mathbb{F}_2$ and $\mathbb{F}_{2^\sigma}$ for PCG-SPDZ. We focus on computational costs since communication overhead of both protocols is sublinear in the circuit size. The ST-PCG of [54] works in field $\mathbb{F}_{2^k}$ and recommends $k = 2$. We let c, t be the PCG compression and weight parameters. In [54], the authors recommend using $c = 5, t = 27$ to withstand the recent attack of [15] for generating both MTs and AMTs. We consider a circuit of size $n = 3^{20}$ (AND gates) and split the computation into $q = 3^4$ “batches”. Thus, we generate q batches of n/q triples. The AMTs are generated by working over $\mathbb{F}_{2^{40}}$ for 40 bits of statistical security. We assume regular noise in all LPN-like assumptions.

We stress that comparing to SPDZ *without* PCGs [50, 42, 51] is not meaningful as the communication cost of generating AMTs without PCGs is prohibitive for SPDZ. For m parties and $n = 3^{20}$ gates, the costs of such protocols scale with $(m^2) \cdot n \cdot \sigma$ bits (times a small constant ≈ 6), hinting at a cost of $\approx 200GB$ communication for two parties, while setup costs are estimated to be about $100\times$ smaller and the overall throughput is estimated to be $\approx 40\%$ slower, even in high-bandwidth settings [54, Table 2]. Thus, we consider three variants of our protocol and PCG-SPDZ: one that does not assume SSD (“vanilla”), one that assumes SSD, and one that assumes DSSD. For SSD we let q (the number of batches in the evaluation) to also be the parameter for which $(n, k, q, \mathbb{F})$ -SSD holds. For DSSD we also let r (the number of repetitions of the same matrix) be the parameter for which $(n, k, q, r, \mathbb{F})$ -DSSD holds. We attribute costs to three sources as these are the only parts of

the protocols:

1. **Generation of MTs:** These costs are identical in our protocol and PCG-SPDZ. In both cases they require a protocol for $\mathcal{F}_{\text{DMPF}}$ and evaluating the ST-PCG linear mappings. DMPF costs are amortized away when (D)SSD is assumed [52, 2].
2. **Authentication costs:** This is the cost of authenticating the triples generated by the ST-PCG. In the case of PCG-SPDZ, AMTs are generated, so we estimate the cost as $\sigma \times$ the cost of generating unauthenticated MTs. Same optimizations apply when (D)SSD is assumed. In our protocol the authentication is achieved by calling $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$. Concretely, we use two-party DPFs to realize $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$, while in the multi-party setting we use multi-party DPFs. We use the three-party DPF of [1]. When no SSD is assumed we use a sum of DPFs to realize $\mathcal{F}_{\text{DMPF}}$ as setup cannot be amortized. We also use $\Pi_{\text{PIR}}^{\text{UV}}$ to realize $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ with balancing optimization in all three variants of our protocol. With no SSD we use balancing parameter $b = 230$ while with (D)SSD we use $b^* = 26$. We take $r = q = 81$ which gives, according to our analysis 99 bits of statistical security.
3. **Verification costs:** This is the cost of verifying that the semi-honest protocol was carried out honestly, given the authentication. In all cases besides our protocol with DSSD this cost is $2n$ multiplications over $\mathbb{F}_{2^\sigma}$. In our protocol with DSSD we use FLIOPs to authenticate the triples, causing an increase in the verification cost to $10n$ multiplications over $\mathbb{F}_{2^\sigma}$.

In Table 1, we neglect low-order terms to provide a cleaner picture of the costs. We consider the total costs in terms of four types of building blocks:

1. $\text{DMPF}(n, w, \mathbb{F})$ - The cost of a maliciously secure 2-party protocol realizing $\mathcal{F}_{\text{DMPF}}$ for m parties, query weight w over field $\mathbb{F}$. This cost disregards setup costs when (D)SSD is assumed. We use $\text{DPF}(n, \mathbb{F})$ to denote $\text{DMPF}(n, 1, \mathbb{F})$.
2. $\text{Lin}(m, \mathbb{F})$ - The cost of the ST-PCG linear compressive mappings (G_A, G_B, G_C) to a vector of size m over field $\mathbb{F}$. Although this building block differs between the ST-PCGs we consider, for fixed m and $\mathbb{F}$ the cost is roughly the same.
3. $\text{Mul}_{\mathbb{F}}$ - The cost of a field multiplication in $\mathbb{F}$.
4. $|\pi_{\text{Mul}}^{\mathbb{F}}|$ - The cost of a *secure* field multiplication in $\mathbb{F}$.

Benchmark Code and Environment. We run all our benchmarks on a 16-core Intel i7-11800H laptop with 32GB of RAM. We use the code of [11]³ for generating MTs and executing the semi-honest protocol, and the code of [54]⁴ for generating AMTs. For the linear mappings of the ST-PCG we use the FFT code of [11] as well. For the DPFs we use the code of [22]⁵ for DPF expansion and estimate maliciously secure DPF setup costs as described below. For the FLIOP-based verification we use the code from [47]⁶. All our benchmarks run on a single core and results are averaged over 50 runs, eliminating top and bottom 10% of the runs to mitigate outliers.

³Available at <https://github.com/sachaservan/FOLEAGE-PCG>

⁴Available at <https://github.com/zhli271828/Trace-F2-OLE-PCG>

⁵Available at <https://github.com/MatanHamilis/dmpf>

⁶Available at <https://github.com/MatanHamilis/PreprocessingForLife>

Benchmark results. In our analysis, we assume $\text{DMPF}(n, w, \mathbb{F}) \approx 3 \cdot \text{DMPF}(n, 1, \mathbb{F})$ under (D)SSD as efficient batching protocols for DMPF exist [2, 52, 21] via Cuckoo hashing. However, without (D)SSD we have $\text{DMPF}(n, w, \mathbb{F}) \approx w \cdot \text{DMPF}(n, 1, \mathbb{F})$. We obtained the following benchmarks: $4c^2t \cdot \text{DMPF}(\frac{n}{tq}, t, \mathbb{F}_4) = 18.4\text{s}$ without (D)SSD, reducing to 2.04s with (D)SSD; $\text{Lin}(\frac{n}{q}, \mathbb{F}_4) = 0.82\text{s}$; and $n \cdot \text{Mul}_{\mathbb{F}_{2^\sigma}} = 2.1\text{s}$. We rely on the estimates of [19] that $|\pi_{\text{Mul}}^{\mathbb{F}_{2^\sigma}}|$ reaches 50,000/s throughput. We summarize our benchmark results in Table 2 from which it follows that even with no SSD optimization our protocol already improves over PCG-SPDZ by $3\times$. With SSD, our protocol achieves an improvement of $1.3\times$, while with DSSD it achieves an improvement of over $5\times$. The results also suggest that future research on secure protocols for $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ can further minimize the overhead compared to the semi-honest protocol. In particular, the high costs associated with PIR query answering and reconstruction suggest that we will likely benefit from PIR with preprocessing [4, 34], while reducing query generation and reconstruction can be achieved via PIR batching techniques [49].

Table 2: Benchmark results of cost factors from Table 1 with our concrete parameters described in Section 7. We note that PCG-SPDZ does not benefit from DSSD over SSD to the best of our knowledge. Colors retain the same meaning as in Table 1. The rightmost column measures the multiplicative overhead over the semi-honest protocol.

Protocol		Semi-Honest	Authentication	Verification	Total	$SH\times$
PCG-SPDZ		1557s	62280s	4.1s	63841s	41
	+ (D)SSD	231s	9265s		9500s	41
This work		1557s	2656s + 6998s + 2835s + 6790s = 19279s		20840s	13.3
	+ SSD	231s	2592s + 764s + 2835s + 767s = 6958s		7193s	31.1
	+ DSSD		32s + 764s + 35s + 767s = 1598s	20.5s	1633s	7.07

The Multiparty Case. In the multiparty case there is much less prior work to compare against concretely, let alone implementations. We stay focused on protocols that incur $o(n)$ communication

Table 3: Benchmark results of cost factors for the $m = 3$ party case. Table format and colorings are similar to Table 2.

Protocol		Semi-Honest	Authentication	Verification	$SH\times$
PCG-SPDZ		3114s	$5.3 \cdot 10^5\text{s}$	4.1s	170
	+ (D)SSD	462s			1148
This work		3114s	2656s + 66339s + 25515s + 70856s = $1.6 \cdot 10^5\text{s}$		51.4
	+ SSD	462s	2656s + 6552s + 25515s + 8857s = 43580s		94
	+ DSSD		98s + 6552s + 945s + 8857s = 16452s	20.5s	35

overhead in n . The PCG-SPDZ analogue for the multiparty case is [1], based on 3-party DPFs, and is significantly more expensive than the 2-party variant, with per-party computational costs of $\Omega(m^2 \cdot N)$ and large constant factors. In our case we generate m -party *unauthenticated* triples, which incurs an $m - 1 \times$ overhead compared to the 2-party case from exactly the same ST-PCGs. Since ST-PCGs over any field exist [54] we focus on realizing $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$. We provide some estimates on the concrete costs for the simplest multiparty case of $m = 3$ parties, assuming the expansion of a 3-party DPF over similar domains as the 2-party DPF takes roughly the same time as the bulk of both constructions’ work is dominated by PRG expansions. We keep the same parameters as in the 2-party case, but use $q = r = 27$, as this suffices for our case. We are not aware of any improvement (besides reduced setup costs) that [1] can make by assuming (D)SSD, due to the lack of concretely efficient multi-party DPFs, let alone DMPFs. All authentication costs not associated with `Lin` grow by a factor of m^2 compared to the 2-party case.

PCG-SPDZ Costs. In [1] a total of $4(mct)^2 q \sigma$ 3P-DPFs of size n/qt over $\mathbb{F}_4$ are evaluated. From our benchmarks only this evaluation will take over 147.4 hours.

Our approach. To simplify comparison, we realize $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ with exactly the same 3-party DPFs used by [1] for the purpose of generating the PIR queries. We assume $|\pi_{\text{Mul}}^{\mathbb{F}_{2^\sigma}}|$ reaches 500/s throughput, as 3-party multiplication is twice as expensive per party according to the estimates of [42], and take balancing parameters $b = 8$ and $b^* = 1$. Unlike the case of 3-party DPFs, under (D)SSD we can reuse the PIR query responses to construct q different linear combinations of the query responses. We achieve a $3\times$, $12\times$, and $33\times$ speedup over PCG-SPDZ with no (D)SSD, SSD, and DSSD, respectively. This significant improvement is mainly due to the lack of existing efficient multiparty DPFs and DMPFs. We note that with DSSD we break the $\sigma\times$ overhead even in the multiparty case!

References

- [1] Damiano Abram and Peter Scholl. Low-communication multiparty triple generation for SPDZ from Ring-LPN. In *Public-Key Cryptography - PKC*, 2022.
- [2] Amit Agarwal, Srinivasan Raghuraman, and Peter Rindal. Fully distributed multi-point functions for PCGs and beyond. Cryptology ePrint Archive, Paper 2025/2294, 2025.
- [3] Donald Beaver. Efficient multiparty protocols using circuit randomization. In *CRYPTO*, 1991.
- [4] Amos Beimel, Yuval Ishai, and Tal Malkin. Reducing the servers’ computation in private information retrieval: PIR with preprocessing. *Journal of Cryptology*, 17(2):125–151, March 2004.
- [5] Mihir Bellare, editor. *CRYPTO 2000*, volume 1880 of *LNCS*. Springer, Berlin, Heidelberg, August 2000.
- [6] Rikke Bendlin, Ivan Damgård, Claudio Orlandi, and Sarah Zakarias. Semi-homomorphic encryption and multiparty computation. In *EUROCRYPT*, 2011.

- [7] Daniel J. Bernstein. The transposition principle. <https://cr.yp.to/transposition.html>, 2024. Accessed: 2024-10-03.
- [8] Alexander R. Block, Hemanta K. Maji, and Hai H. Nguyen. Secure computation with constant communication overhead using multiplication embeddings. In *INDOCRYPT*, 2018.
- [9] Avrim Blum, Merrick Furst, Michael Kearns, and Richard J Lipton. Cryptographic primitives based on hard learning problems. In *Annual international cryptology conference*, pages 278–291. Springer, 1993.
- [10] Avrim Blum, Adam Kalai, and Hal Wasserman. Noise-tolerant learning, the parity problem, and the statistical query model. *Journal of the ACM (JACM)*, 50(4):506–519, 2003.
- [11] Maxime Bombar, Dung Bui, Geoffroy Couteau, Alain Couvreur, Clément Ducros, and Sacha Servan-Schreiber. FOLEAGE: $\mathbb{F}_4$ OLE-based multi-party computation for boolean circuits. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 69–101. Springer, 2024.
- [12] Maxime Bombar, Geoffroy Couteau, Alain Couvreur, and Clément Ducros. Correlated pseudorandomness from the hardness of quasi-abelian decoding. In *Advances in Cryptology - CRYPTO*, 2023.
- [13] Maxime Bombar, Geoffroy Couteau, Alain Couvreur, and Clément Ducros. Correlated pseudorandomness from the hardness of quasi-abelian decoding. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part IV*, volume 14084 of *LNCS*, pages 567–601. Springer, Cham, August 2023.
- [14] Dan Boneh, Elette Boyle, Henry Corrigan-Gibbs, Niv Gilboa, and Yuval Ishai. Zero-knowledge proofs on secret-shared data via fully linear PCPs. In *CRYPTO*, 2019.
- [15] Charles Bouillaguet, Claire Delaplace, Mickaël Hamdad, and Damien Vergnaud. Practical cryptanalysis of pseudorandom correlation generators based on quasi-abelian syndrome decoding. *Cryptology ePrint Archive*, 2025.
- [16] Elette Boyle, Geoffroy Couteau, Niv Gilboa, and Yuval Ishai. Compressing vector OLE. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018*, pages 896–912. ACM Press, October 2018.
- [17] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators: Silent OT extension and more. In *CRYPTO*, 2019.
- [18] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators: Silent OT extension and more. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part III*, volume 11694 of *LNCS*, pages 489–518. Springer, Cham, August 2019.
- [19] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators from Ring-LPN. In *CRYPTO*, 2020.

- [20] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators from ring-LPN. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part II*, volume 12171 of *LNCS*, pages 387–416. Springer, Cham, August 2020.
- [21] Elette Boyle, Niv Gilboa, Matan Hamilis, Yuval Ishai, and Yaxin Tu. Improved constructions for distributed multi-point functions. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 2414–2432, 2025.
- [22] Elette Boyle, Niv Gilboa, Matan Hamilis, Yuval Ishai, and Yaxin Tu. Improved constructions for distributed multi-point functions. In *2025 IEEE Symposium on Security and Privacy*, pages 2414–2432. IEEE Computer Society Press, May 2025.
- [23] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 337–367. Springer, 2015.
- [24] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 337–367. Springer, Berlin, Heidelberg, April 2015.
- [25] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing: Improvements and extensions. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016*, pages 1292–1303. ACM Press, October 2016.
- [26] Elette Boyle, Niv Gilboa, and Yuval Ishai. Secure computation with preprocessing via function secret sharing. In *TCC*, 2019.
- [27] Elette Boyle, Niv Gilboa, Yuval Ishai, and Ariel Nof. Sublinear gmw-style compiler for MPC with preprocessing. In *CRYPTO*, 2021.
- [28] Elette Boyle, Niv Gilboa, Yuval Ishai, and Ariel Nof. Secure multiparty computation with sublinear preprocessing. In *EUROCRYPT*, 2022.
- [29] Pierre Briaud and Morten Øygarden. A new algebraic approach to the regular syndrome decoding problem and implications for pcg constructions. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 391–422. Springer, 2023.
- [30] Sai Sheshank Burra, Enrique Larraia, Jesper Buus Nielsen, Peter Sebastian Nordholt, Claudio Orlandi, Emmanuela Orsini, Peter Scholl, and Nigel P. Smart. High-performance multi-party computation for binary circuits based on oblivious transfer. *J. Cryptol.*, 2021.
- [31] Dario Catalano, Mario Di Raimondo, Dario Fiore, and Irene Giacomelli. Monz2ka: Fast maliciously secure two party computation on zk. *IACR Cryptology ePrint Archive*, 2019.
- [32] Benny Chor, Oded Goldreich, Eyal Kushilevitz, and Madhu Sudan. Private information retrieval. In *36th FOCS*, pages 41–50. IEEE Computer Society Press, October 1995.
- [33] Benny Chor, Eyal Kushilevitz, Oded Goldreich, and Madhu Sudan. Private information retrieval. *J. ACM*, 45(6):965–981, November 1998.

- [34] Henry Corrigan-Gibbs and Dmitry Kogan. Private information retrieval with sublinear online time. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part I*, volume 12105 of *LNCS*, pages 44–75. Springer, Cham, May 2020.
- [35] Nicolas Courtois, Alexander Klimov, Jacques Patarin, and Adi Shamir. Efficient algorithms for solving overdefined systems of multivariate polynomial equations. In *International Conference on the Theory and Applications of Cryptographic Techniques*, pages 392–407. Springer, 2000.
- [36] Geoffroy Couteau, Peter Rindal, and Srinivasan Raghuraman. Silver: Silent VOLE and oblivious transfer from hardness of decoding structured LDPC codes. In *CRYPTO*, 2021.
- [37] Ronald Cramer, Ivan Damgård, Daniel Escudero, Peter Scholl, and Chaoping Xing. Spdz2k: Efficient MPC mod 2k for dishonest majority. In *CRYPTO*, 2018.
- [38] Ivan Damgård, Marcel Keller, Enrique Larraia, Valerio Pastro, Peter Scholl, and Nigel P. Smart. Practical covertly secure MPC for dishonest majority - or: Breaking the SPDZ limits. In *ESORICS*, 2013.
- [39] Ivan Damgård, Jesper Buus Nielsen, Michael Nielsen, and Samuel Ranellucci. The tinytable protocol for 2-party secure computation, or: Gate-scrumbling revisited. In *CRYPTO*, 2017.
- [40] Ivan Damgård, Valerio Pastro, Nigel P. Smart, and Sarah Zakarias. Multiparty computation from somewhat homomorphic encryption. In *CRYPTO*, 2012.
- [41] Ivan Damgård and Sarah Zakarias. Constant-overhead secure computation of boolean circuits using preprocessing. In *TCC*, 2013.
- [42] Tore Kasper Frederiksen, Marcel Keller, Emmanuela Orsini, and Peter Scholl. A unified approach to MPC with preprocessing using OT. In *Proceedings, Part I, of the 21st International Conference on Advances in Cryptology – ASIACRYPT 2015*, 2015.
- [43] Daniel Genkin, Yuval Ishai, Manoj Prabhakaran, Amit Sahai, and Eran Tromer. Circuits resilient to additive attacks with applications to secure computation. In *STOC*, 2014.
- [44] Daniel Genkin, Yuval Ishai, Manoj M. Prabhakaran, Amit Sahai, and Eran Tromer. Circuits resilient to additive attacks with applications to secure computation. In *Proceedings of the Forty-Sixth Annual ACM Symposium on Theory of Computing*, STOC ’14, page 495–504, New York, NY, USA, 2014. Association for Computing Machinery.
- [45] Oded Goldreich. *The Foundations of Cryptography - Volume 2, Basic Applications*. Cambridge University Press, 2004.
- [46] Oded Goldreich, Silvio Micali, and Avi Wigderson. How to play any mental game or A completeness theorem for protocols with honest majority. In *STOC*, 1987.
- [47] Matan Hamilis, Elette Boyle, Niv Gilboa, Yuval Ishai, and Ariel Nof. Preprocessing for Life: Dishonest-Majority MPC with a Trusted or Untrusted Dealer . In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 41–41, Los Alamitos, CA, USA, May 2025. IEEE Computer Society.

- [48] Yuval Ishai, Eyal Kushilevitz, Sigurd Meldgaard, Claudio Orlandi, and Anat Paskin-Cherniavsky. On the power of correlated randomness in secure computation. In *TCC*, 2013.
- [49] Yuval Ishai, Eyal Kushilevitz, Rafail Ostrovsky, and Amit Sahai. Batch codes and their applications. In László Babai, editor, *36th ACM STOC*, pages 262–271. ACM Press, June 2004.
- [50] Marcel Keller, Emmanuela Orsini, and Peter Scholl. MASCOT: faster malicious arithmetic secure computation with oblivious transfer. In *ACM CCS*, 2016.
- [51] Marcel Keller, Valerio Pastro, and Dragos Rotaru. Overdrive: Making SPDZ great again. In *EUROCRYPT*, 2018.
- [52] Vladimir Kolesnikov, Stanislav Peceny, Srinivasan Raghuraman, and Peter Rindal. Stationary syndrome decoding for improved PCGs. Cryptology ePrint Archive, Paper 2025/295, 2025.
- [53] E. Kushilevitz and R. Ostrovsky. Replication is not needed: single database, computationally-private information retrieval. In *Proceedings 38th Annual Symposium on Foundations of Computer Science*, pages 364–373, 1997.
- [54] Zhe Li, Chaoping Xing, Yizhou Yao, and Chen Yuan. Efficient pseudorandom correlation generators for any finite field. Springer-Verlag, 2025.
- [55] Jesper Buus Nielsen, Peter Sebastian Nordholt, Claudio Orlandi, and Sai Sheshank Burra. A new approach to practical active-secure two-party computation. In *CRYPTO*, 2012.
- [56] Eugene Prange. The use of information sets in decoding cyclic codes. *IRE Transactions on Information Theory*, 8(5):5–9, 1962.
- [57] Daniel Salmond, Alex Grant, Ian Grivell, and Terence Chan. On the rank of random matrices over finite fields. *arXiv preprint arXiv:1404.3250*, 2014.
- [58] Jacques Stern. A method for finding codewords of small weight. In *International colloquium on coding theory and applications*, pages 106–113. Springer, 1988.

A MPC Security Definition

We consider the standard m -party setting of (computational) “security with abort” in the presence of a malicious adversary who may corrupt any number of parties. We assume that all parties are connected via point-to-point secure channels, which enable them to send messages to each other. We consider functionalities represented by arithmetic circuits over a finite field $\mathbb{F}$. For $\mathbb{F} = \mathbb{F}_2$, this captures Boolean circuits.

We start by defining the standard syntax of a protocol.

Definition A.1 (Protocol). *Let $\mathcal{F}$ be a family of descriptions of n -party (deterministic or randomized) functionalities. A protocol Π for $\mathcal{F}$ is defined by a polynomial-time algorithm NextMsg , which given $(1^\kappa, f, j, i, x_i, r_i, \vec{m})$ outputs a vector of n messages sent by P_i in round j , based on the security parameter κ , function description $f \in \mathcal{F}$, local input x_i of P_i , local randomness r_i of P_i , and vector $\vec{m}$ of messages received by P_i in previous rounds. If the output is of the form (out, y) , then P_i outputs y and halts.*

In this work, we will typically consider $\mathcal{F}$ to be the class of arithmetic circuits (with addition and multiplication gates) over a finite ring R , where R can either be fixed or be given as part of the circuit description. In particular, the case of Boolean circuits is captured by $R = \mathbb{F}_2$. We use the standard ideal/real world definition from [45, 5], which we outline here for completeness. Since we consider the case of a dishonest majority, we will need to settle for *computational security*.

Let $\mathcal{A}$ be an adversary who chooses a set of parties before the beginning of the execution and corrupts them. We distinguish between a (passive) *semi-honest* adversary, who sends messages according to the protocol but observes the secrets and messages of the corrupted parties, and an (active) *malicious* adversary, who can also choose the messages sent by the corrupted parties. We assume that the adversary is *rushing*, namely, it first receives the messages sent by the honest parties in each round, and only then determines the corrupted parties' messages in this round.

To formally define security, let $\text{REAL}_{\Pi, \mathcal{A}, T}(1^\kappa, f, \vec{v})$ be a random variable that consists of the view of the adversary $\mathcal{A}$ controlling a set of parties T , and the honest parties' outputs, following an execution of a protocol Π on security parameter κ , function description $f \in \mathcal{F}$, and input vector⁷ $\vec{v}$.

We similarly define an ideal-world execution with an ideal-world adversary $\mathcal{S}$, where $\mathcal{S}$ and the honest parties interact with a trusted party who computes f for them. Our results can either be cast in the setting of security *with selective abort*, where the adversary can decide which subset of the honest parties receive the output, or (assuming either ideal broadcast or PKI) in the standard setting of *security with abort*, where either all honest parties abort or all receive the output. These two variants are captured by allowing $\mathcal{S}$, after receiving the outputs of the corrupted parties, to send an **abort** command to the trusted party that causes either a (specified) subset or all of the uncorrupted parties to output a special $\perp$ symbol. We denote by $\text{IDEAL}_{\mathcal{F}, \mathcal{S}, T}(1^\kappa, f, \vec{v})$ the random variable that consists of the output of $\mathcal{S}$ and the honest parties in this ideal execution on security parameter κ , function description $f \in \mathcal{F}$, and input vector $\vec{v}$, where $\mathcal{S}$ controls a set of parties T . The security definition states that a protocol Π realizes f (with computational security) if for every efficient real-world adversary $\mathcal{A}$ there exists an efficient ideal-world adversary $\mathcal{S}$, such that the two random variables REAL and IDEAL are computationally indistinguishable. Formally:

Definition A.2 (Secure MPC). *Let $\mathcal{F}$ be a family of representations of n -party functionalities. We say that a protocol Π securely realizes $\mathcal{F}$, if for every set $T \subset [n]$ of corrupted parties the following holds. For every malicious PPT real-world adversary $\mathcal{A}$ controlling parties in T , there exists a PPT ideal-world adversary $\mathcal{S}$ controlling the same parties, such that for every $\text{poly}(\kappa)$ -size sequence $x_\kappa = (1^\kappa, f_\kappa, \vec{v}_\kappa)$, the distribution ensembles $\text{REAL}_{\Pi, \mathcal{A}, T}(x_\kappa)$ and $\text{IDEAL}_{\mathcal{F}, \mathcal{S}, T}(x_\kappa)$ are computationally indistinguishable. We say that Π realizes $\mathcal{F}$ with semi-honest security if the above holds for every semi-honest $\mathcal{A}$ and $\mathcal{S}$.*

Hybrid models. To enable a more modular description and analysis of protocols, we use a *hybrid model* in which parties have access to a trusted party computing a specific “helper” subfunctionality. The modular sequential composition theorem of [5] states that it is possible to replace the trusted party computing the subfunctionality with a real protocol realizing the subfunctionality. When the subfunctionality is g , we say that the protocol works in the g -hybrid model. When considering protocols in hybrid models, we can strengthen the notion of computational security and refer to statistical or even perfect security, which is defined similarly except for allowing $\mathcal{A}, \mathcal{S}$ to be com-

⁷To support (sequential) composition, the vector $\vec{v}$ should also include additional *auxiliary inputs* which we omit from the notation for simplicity.

putationally unbounded and replacing computational indistinguishability by statistical or perfect indistinguishability.

B Secret Sharing and Authenticated Secret Sharing

In this section we provide definitions for secret sharing and authenticated secret sharing schemes.

Definition B.1 (Secret Sharing). *An m -of- m secret sharing scheme for a set S is a pair of algorithms $(\text{Share}, \text{Rec})$ such that:*

- *$\text{Share}(v)$ is a randomized method such that, given a secret value $v \in S$, it outputs m shares $\llbracket v \rrbracket = (v_1, \dots, v_m)$, where the i -th party is given share $v_i \in \{0, 1\}^\ell$.*
- *$\text{Rec}(v_1, \dots, v_m)$ Given values $v_1, \dots, v_m \in \{0, 1\}^\ell$, outputs an element from $S \cup \{\perp\}$.*

The algorithms Share and Rec must satisfy the following:

- *Correctness: For any value $v \in S$, a sharing of v always reconstructs to v , formally:*

$$\Pr \left[v' = v \mid \begin{array}{ll} (v_1, \dots, v_m) & \leftarrow \text{Share}(v) \\ v' & \leftarrow \text{Rec}(v_1, \dots, v_m) \end{array} \right] = 1$$

where probability is taken over random coins used by the Share algorithm.

- *Privacy: There exists a simulator Sim which may optionally take a (computational or statistical) security parameter, or none if the scheme guarantees perfect privacy, and simulates the joint distribution of shares of any subset of parties of size at most $m - 1$.*

Definition B.2 (Authenticated Secret Sharing). *An ε -secure authenticated secret sharing scheme is a secret sharing scheme that also satisfies the following authentication property:*

- *Authentication: For any adversary $\mathcal{A}$ corrupting a set $T \subsetneq [m]$ it holds that for any v :*

$$\Pr \left[y' \notin \{v, \perp\} \mid \begin{array}{ll} (v_1, \dots, v_m) & \leftarrow \text{Share}(v) \\ (v_i^*)_{i \in T} & \leftarrow \mathcal{A}((v_i)_{i \in T}) \\ y' & \leftarrow \text{Rec}((v_i)_{i \notin T}, (v_i^*)_{i \in T}) \end{array} \right] \leq \varepsilon$$

where probability is taken over random coins used in Share , and where the outcome of whether $y' = v$ or $\perp$ is efficiently simulatable given the view of corrupted parties (but not given v). In the case of computational authenticity, we require $\mathcal{A}$ to be computationally bounded.

C DSSD Security

Consider a DSSD instance in which $\lceil q/r \rceil$ uniformly random matrices are sampled from $\mathbb{F}^{n \times k}$: $\mathbf{G}_1, \dots, \mathbf{G}_{\lceil q/r \rceil} \in_R \mathbb{F}^{n \times k}$, and q sparse vectors are sampled from $\mathbb{F}^k$: $\mathbf{e}_1, \dots, \mathbf{e}_q \in_R \mathcal{L}$. Assume that $\mathbf{e}_1, \dots, \mathbf{e}_q$ are all t -sparse. Sample q vectors $\mathbf{x}_1, \dots, \mathbf{x}_q \in \mathbb{F}^n$ uniformly at random and independently.

Partition the vectors $\mathbf{e}_1, \dots, \mathbf{e}_q$ into $\lceil q/r \rceil$ subsets of size r (except maybe for the last set), and view each set as the columns of a matrix. The result is $\lceil q/r \rceil$ matrices $\mathbf{E}_1, \dots, \mathbf{E}_{\lceil q/r \rceil}$, which are of

dimensions $k \times r$ (except possibly for the last which might have fewer columns). Let $\mathbf{F}_j = \mathbf{G}_j \mathbf{E}_j$ for all $j = 1, \dots, \lceil q/r \rceil$. Let $\mathbf{F}$ be the concatenation of all $\mathbf{F}_j$ into a single $n \times q$ matrix and let $\mathbf{X}$ be the $n \times q$ matrix with columns $\mathbf{x}_1, \dots, \mathbf{x}_q$.

The DSSD assumption states that $\mathbf{F}$ and $\mathbf{X}$ are computationally indistinguishable.

We begin the security analysis of DSSD by considering the rank of $\mathbf{F}$. Obviously, if the expected rank of $\mathbf{F}$ is different from the expected rank of $\mathbf{X}$ then the DSSD assumption is broken. As stated previously, if $r > t$ then $\mathbf{F}$ does not have full rank, and since as we show $\mathbf{X}$ does have full rank with high probability, the DSSD assumption is false.

An important tool in our analysis is Lemma C.1, which is a standard result on the probability that a uniformly random matrix over a finite field $\mathbb{F}$ is full-rank.

Lemma C.1. *Let $\mathbb{F}$ be a finite field, let $n, k, n \leq k$ be positive integers, and let $\mathbf{A} \in \mathbb{F}^{n \times k}$ be a matrix in which each entry is sampled randomly and independently from $\mathbb{F}$. Then, the probability that $\mathbf{A}$ has full rank is*

$$\prod_{i=k-n+1}^k \left(1 - \frac{1}{|\mathbb{F}|^i}\right).$$

The proof of Lemma C.1 appears in the literature, e.g. in [57], Lemma 1.

The analysis in the next theorem assumes some fairly minor restrictions on the relations of the parameters in the DSSD assumption. First, note that while $t \ll k$ in any setting of Syndrome Decoding, since the vectors $\mathbf{e}_i$ are sparse, all applications of SD for PCGs, including in this work, require further that $t \ll n$. Otherwise, the PCG seed length, which is $\omega(t)$ is at least as large as the circuit size, thus achieving no compression of the correlated randomness. In what follows, it is sufficient to assume that $3 \lceil q/r \rceil t < n$. For a similar reason $q < n$ in applications of SSD to PCGs. We also assume that $r \geq 2$, since otherwise DSSD coincides with SSD.

Remark C.1. *In the interest of slightly simplifying the assumptions for Theorem C.1, we assume that the distribution $\mathcal{G}$ from which the matrices $\mathbf{G}_1, \dots, \mathbf{G}_{\lceil q/r \rceil}$ are sampled is the uniform distribution on $\mathbb{F}^{n \times k}$. The theorem can be generalized to other distributions by considering the following experiment. Sample $\mathbf{G}_1, \dots, \mathbf{G}_{\lceil q/r \rceil} \in_R \mathcal{G}$ and let $I = \{i_1, \dots, i_t\} \subseteq \{1, \dots, k\}$ be a random set of t indices. For all $j = 1, \dots, \lceil q/r \rceil$ let $\mathbf{G}'_j$ be the $n \times t$ sub-matrix of $\mathbf{G}_j$ that is made up of the t columns at the positions indicated by I , and let $\mathbf{G}'$ be the concatenation of all $\mathbf{G}'_j$. If the probability that $\mathbf{G}'$ has full rank, i.e. has rank $t \lceil q/r \rceil$, is at least $(e/2)^{\frac{-r}{x}}$ then the statement of the theorem holds. To provide some intuition on why this is the case, note that if I indicates the noisy positions of the sparse vectors $\mathbf{e}_i$ then $\mathbf{G}'$ includes exactly all the columns of $\mathbf{G}_1, \dots, \mathbf{G}_{\lceil q/r \rceil}$ whose linear combinations from the vectors $\mathbf{G}_j \mathbf{e}_i$ should be indistinguishable from random vectors to satisfy DSSD.*

Theorem C.1 gives a bound on the probability that the ranks of $\mathbf{F}$ and $\mathbf{X}$ are different.

Theorem C.1. *Let σ be a statistical security parameter, let $\mathcal{G}$ be the uniform distribution over $\mathbb{F}^{n \times k}$ and assume that $2 \leq r \leq t$, $3t \lceil q/r \rceil + q < n$ and*

$$t - r \geq \frac{1}{\ln |\mathbb{F}|} \left(\ln r - \ln \left(\ln \frac{3 \lceil q/r \rceil}{3 \lceil q/r \rceil - 2^{-\sigma}} \right) \right).$$

Then, the ranks of $\mathbf{F}$ and $\mathbf{X}$ are identical with probability at least $1 - 2^{-\sigma}$.

Proof. The probability that the ranks of $\mathbf{F}$ and $\mathbf{X}$ are equal is at least the probability that they are both full rank, i.e., of rank q , since they have the same dimensions $n \times q$ and $q < n$.

We begin with obtaining a lower bound on the probability that $\mathbf{F}$ is full rank. If $\mathbf{F}$ is not full rank and the columns of $\mathbf{F}$ are denoted $\mathbf{f}_1, \dots, \mathbf{f}_q$ then there exist $(\alpha_1, \dots, \alpha_q) \in \mathbb{F}^q \setminus \{0^q\}$ such that $\sum_{i=1}^q \alpha_i \mathbf{f}_i = 0$ or in other words, the columns of $\mathbf{F}$ are linearly dependent. The analysis of the probability that $\sum_{i=1}^q \alpha_i \mathbf{f}_i = 0$ is divided into two cases. In the first case, the columns of some sub-matrix $\mathbf{F}_{\mathbf{j}}$ are linearly dependent, while in the second case the columns of all $\mathbf{F}_{\mathbf{j}}, j = 1, \dots, \lceil q/r \rceil$, are linearly independent, i.e. all these sub-matrices are full rank.

For the first case, recall that $\mathbf{F}_{\mathbf{j}} = \mathbf{G}_{\mathbf{j}} \mathbf{E}_{\mathbf{j}}$. $\mathbf{E}_{\mathbf{j}}$ is a $k \times r$ matrix in which there are exactly t rows which are the noisy locations of the vectors $\mathbf{e}_i$. Every entry in one of these rows is chosen randomly and independently from $\mathbb{F}$, while every other entry in $\mathbf{E}_{\mathbf{j}}$ is 0. Therefore, the rank of $\mathbf{E}_{\mathbf{j}}$ is equal to the rank of its $t \times r$ sub-matrix $\mathbf{E}'_{\mathbf{j}}$ consisting of all the randomly chosen entries.

By Lemma C.1 the probability that $\mathbf{E}'_{\mathbf{j}}$ is full rank, i.e. has rank r , is

$$\prod_{i=t-r+1}^t \left(1 - \frac{1}{|\mathbb{F}|^i}\right) \geq \left(1 - \frac{1}{|\mathbb{F}|^{t-r+1}}\right)^r = \left[\left(1 - \frac{1}{|\mathbb{F}|^{t-r+1}}\right)^{|\mathbb{F}|^{t-r+1}}\right]^{\frac{r}{|\mathbb{F}|^{t-r+1}}} \quad (6)$$

To further simplify the bound in Equation 6 we use the fact that $(1 - 1/x)^{x-1} \geq e^{-1}$ for all $x > 1$. This fact follows from elementary calculus since it is equivalent to taking the natural logarithm from both sides and rearranging terms obtaining $\ln(1 - 1/x) + 1/(x-1) \geq 0$ for $x > 1$. Changing variables to $y = 1/x$ we have that the original inequality is satisfied if $\ln(1-y) + y/(1-y) \geq 0$ for all $0 < y < 1$. The function $g(y) = \ln(1-y) + y/(1-y)$ is strictly increasing in the interval $0 < y < 1$ since its derivative is $g'(y) = \frac{y}{(1-y)^2}$ which is greater than zero over the interval. Since $g(0) = 0$ we have that $g(y) > 0$ for all $0 < y < 1$ proving that $(1 - 1/x)^{x-1} \geq e^{-1}$ for all $x > 1$.

Since $t \geq r$ and $|\mathbb{F}| \geq 2$ we have that $|\mathbb{F}|^{t-r+1} \geq 2$ implying that the above fact holds for $x = |\mathbb{F}|^{t-r+1}$. It follows that the probability that $\mathbf{E}'_{\mathbf{j}}$ has full rank and therefore that $\mathbf{E}_{\mathbf{j}}$ has full rank is at least

$$\left[\left(1 - \frac{1}{x}\right)^x\right]^{\frac{r}{x}} \geq \left[e^{-1} \left(1 - \frac{1}{x}\right)\right]^{\frac{r}{x}} = \left[\left(\frac{e \cdot x}{x-1}\right)^{-1}\right]^{\frac{r}{x}} \geq (e)^{\frac{-r}{x}}.$$

and the probability that $\mathbf{E}_{\mathbf{j}}$ does not have full rank is at most $1 - e^{\frac{-r}{x}}$. By the union bound, the probability that *any* $\mathbf{E}_{\mathbf{j}}, j = 1, \dots, \lceil q/r \rceil$ does not have full rank is at most $\lceil q/r \rceil \left(1 - e^{\frac{-r}{x}}\right)$.

Let $\mathbf{G}'_{\mathbf{j}}$ be an $n \times t$ sub-matrix of $\mathbf{G}_{\mathbf{j}}$ which contains all the columns whose indices match the row indices of $\mathbf{E}'_{\mathbf{j}}$ as a sub-matrix of $\mathbf{E}_{\mathbf{j}}$. It follows that $\mathbf{F}_{\mathbf{j}} = \mathbf{G}_{\mathbf{j}} \mathbf{E}_{\mathbf{j}} = \mathbf{G}'_{\mathbf{j}} \mathbf{E}'_{\mathbf{j}}$.

The probability that all $\mathbf{G}'_{\mathbf{j}}, j = 1, \dots, \lceil q/r \rceil$ have full rank is at least the probability that the matrix $\mathbf{G}' = \mathbf{G}'_1 || \dots || \mathbf{G}'_{\lceil q/r \rceil}$, which is the concatenation of all the matrices $\mathbf{G}'_{\mathbf{j}}$, has full rank. $\mathbf{G}'$ has n rows and $t \lceil q/r \rceil$ columns, and is therefore full rank if it has rank $t \lceil q/r \rceil$.

The same argument used previously to bound the probability that $\mathbf{E}'_{\mathbf{j}}$ is full rank shows that the probability that $\mathbf{G}'$ is full rank is:

$$\prod_{i=n-t\lceil q/r \rceil+1}^n \left(1 - \frac{1}{|\mathbb{F}|^i}\right) \geq (e)^{\frac{-t\lceil q/r \rceil}{|\mathbb{F}|^{n-t\lceil q/r \rceil+1}}}$$

We bound the probability that $\mathbf{G}'$ is full rank by proving that this probability is at least the probability that $\mathbf{E}'_j$ is full rank. It is sufficient to show that $-t \lceil q/r \rceil / |\mathbb{F}|^{n-t\lceil q/r \rceil+1} \geq -r/|\mathbb{F}|^{t-r+1}$ or that $r|\mathbb{F}|^{n-t(\lceil q/r \rceil+1)+r} \geq t \lceil q/r \rceil$. But,

$$r|\mathbb{F}|^{n-t(\lceil q/r \rceil+1)+r} \geq 2^{n-t(2\lceil q/r \rceil)+r} \geq 2^{t\lceil q/r \rceil} \geq t(\lceil q/r \rceil),$$

since $1 \leq r \leq q$, $3t(\lceil q/r \rceil) \leq n$ and $|\mathbb{F}| \geq 2$.

Next, we argue that if both $\mathbf{G}'_j$ and $\mathbf{E}'_j$ are full rank, then $\mathbf{F}_j$ is also full rank. Since $\mathbf{G}'_j$ is of dimensions $n \times t$, $t < n$, it has rank t implying that it has a regular sub-matrix of dimensions $t \times t$. The product of that sub-matrix and $\mathbf{E}'_j$, which has dimensions $t \times r$, $r < t$, is of rank r , meaning that $\mathbf{F}_j$ is of full rank.

The probability that $\mathbf{F}_j$ is not full rank for some j is therefore at most the probability that $\mathbf{E}'_j$ is not full rank for some j or that $\mathbf{G}'$ is not full rank. By the union bound that probability is at most $(\lceil q/r \rceil + 1)(1 - e^{-r/|\mathbb{F}|^{t-r+1}})$.

In the second case, $\mathbf{F}$ is not full rank, i.e. it has a subset of linearly dependent column vectors, but not all these columns are part of the same sub-matrix $\mathbf{F}_j$. In other words, there is a $j^* \in \{1, \dots, \lceil q/r \rceil\}$ such that if the column vectors of $\mathbf{F}_{j^*}$ are denoted $\mathbf{f}'_1, \dots, \mathbf{f}'_r$ and all other column vectors of $\mathbf{F}$ are denoted $\mathbf{f}''_1, \dots, \mathbf{f}''_{q-r}$ then $\sum_{i=1}^{q-r} \alpha_i \mathbf{f}''_i = \sum_{i=1}^r \beta_i \mathbf{f}'_i$ for two non-zero sequences of field elements $(\alpha_1, \dots, \alpha_{q-r})$ and $(\beta_1, \dots, \beta_r)$.

Since $\mathbf{F}_j = \mathbf{G}'_j \mathbf{E}'_j$ every column of $\mathbf{F}_j$ is a linear combination of the t columns of $\mathbf{G}'_j$. Denote the columns of $\mathbf{G}'_{j^*}$ by $\mathbf{g}'_1, \dots, \mathbf{g}'_t$ and denote the other columns of $\mathbf{G}' = \mathbf{G}'_1 \parallel \dots \parallel \mathbf{G}'_{\lceil q/r \rceil}$ by $\mathbf{g}''_1, \dots, \mathbf{g}''_{(\lceil q/r \rceil-1)t}$. Then, $\sum_{i=1}^{(\lceil q/r \rceil)t} \alpha'_i \mathbf{g}'_i = \sum_{i=1}^t \beta'_i \mathbf{g}'_i$ for non-zero sequences of field elements $(\alpha'_1, \dots, \alpha'_{\lceil q/r \rceil(t-1)})$ and $(\beta'_1, \dots, \beta'_t)$. In other words, in the second case, $\mathbf{G}'$ is not full rank.

Therefore, if $\mathbf{E}'_1, \dots, \mathbf{E}'_{\lceil q/r \rceil}$ and $\mathbf{G}'$ are all full rank then $\mathbf{F}$ is also full rank. It follows that the probability that $\mathbf{F}$ is not full rank is at most $(\lceil q/r \rceil + 1)(1 - e^{-r/|\mathbb{F}|^{t-r+1}})$.

On the other hand, the probability that $\text{rank}(\mathbf{X}) < \text{rank}(\mathbf{F})$ is at most the probability that $\mathbf{X}$ is not full rank. $\mathbf{X}$ is a random matrix of dimensions $n \times q$ over $\mathbb{F}$ and by Lemma C.1 the probability that it is full rank is at least $\prod_{i=n-q+1}^n \left(1 - \frac{1}{|\mathbb{F}|^i}\right)$, which is at least $e^{-q/|\mathbb{F}|^{n-q+1}}$ by the same argument used earlier. We prove that the probability that $\mathbf{X}$ is full rank is at least the probability that $\mathbf{E}'_j$ is full rank by showing that $e^{-q/|\mathbb{F}|^{n-q+1}} \geq e^{-r/|\mathbb{F}|^{t-r+1}}$. It is sufficient to prove that $-q/|\mathbb{F}|^{n-q+1} \geq -r/|\mathbb{F}|^{t-r+1}$ or that $q/r \leq |\mathbb{F}|^{n-q-(t-r)}$. But $n - q - (t - r) \geq n - q - t \geq q + 3 \lceil \frac{q}{r} \rceil t - q - t$, by the assumption on n . Therefore $n - q - (t - r) \geq \lceil \frac{q}{r} \rceil$ and $|\mathbb{F}|^{n-q-(t-r)} \geq q/r$.

To summarize the above arguments, the probability that the ranks of $\mathbf{F}$ and $\mathbf{X}$ are equal is at least the probability that all the matrices $\mathbf{E}'_1, \dots, \mathbf{E}'_{\lceil q/r \rceil}$, $\mathbf{G}'$ and $\mathbf{X}$ are full rank. Conversely, the probability that the ranks of $\mathbf{F}$ and $\mathbf{X}$ are *not* equal is at most the probability that one of $\mathbf{E}'_1, \dots, \mathbf{E}'_{\lceil q/r \rceil}$, $\mathbf{G}'$, $\mathbf{X}$ is not full rank, which by the union bound is at most $(\lceil q/r \rceil + 2) \left(1 - e^{-r/|\mathbb{F}|^{t-r+1}}\right)$, which is equal to or less than $3 \lceil q/r \rceil \left(1 - e^{-r/|\mathbb{F}|^{t-r+1}}\right)$.

To prove that this probability is at most $2^{-\sigma}$ note that

$$3 \left\lceil \frac{q}{r} \right\rceil \left(1 - e^{-r/|\mathbb{F}|^{t-r+1}}\right) \leq 2^{-\sigma}$$

is equivalent to

$$\frac{3 \lceil q/r \rceil}{3 \lceil q/r \rceil - 2^{-\sigma}} \geq e^{r/|\mathbb{F}|^{t-r+1}}.$$

Taking the natural logarithm from both sides we have that

$$|\mathbb{F}|^{t-r+1} \geq \frac{r}{\ln \frac{3^{\lceil q/r \rceil}}{3^{\lceil q/r \rceil} - 2^{-\sigma}}},$$

and therefore that

$$t - r + 1 \geq \frac{1}{\ln |\mathbb{F}|} \left(\ln r - \ln \left(\ln \frac{3^{\lceil q/r \rceil}}{3^{\lceil q/r \rceil} - 2^{-\sigma}} \right) \right),$$

which holds due to the statement of the theorem. $\square$

Corollary C.1. *Using the parameters of the concrete use case for our construction in Section 7: $|\mathbb{F}| = 4$, $t = 135$ and $r = q = 81$ in Theorem C.1 we get that DSSD has 99 bits of statistical security against the rank attack, i.e. the probability that the rank of the matrices $\mathbf{F}$ and $\mathbf{X}$ is different is less than 2^{-102} .*

The next class of attacks we consider is the so-called *linear test framework* [36], which covers most of the known attacks against SD and its variants. The framework is defined for the problem of Linear Parity with Noise (LPN) [9], in which we consider a pair $(\mathbf{A}, \mathbf{A}\mathbf{s} + \mathbf{e})$, where $A \in \mathbb{F}^{k \times m}$, $m > n$ is public, $s \in_R \mathbb{F}^m$ is secret and $\mathbf{e} \in_R \mathcal{L} \subseteq \mathbb{F}^k$ is secret and sparse. The goal of the adversary is to distinguish between $(\mathbf{A}, \mathbf{A}\mathbf{s} + \mathbf{e})$ and $(\mathbf{A}, b)$ for $b \in_R \mathbb{F}^k$.

LPN and SD are equivalent (or the dual of one another) since if $\mathbf{G} \in \mathbb{F}^{n \times k}$ is the generator matrix of a linear code and $\mathbf{A} \in \mathbb{F}^{k \times m}$ is its parity check matrix then $\mathbf{G}((\mathbf{A}s) + \mathbf{e}) = \mathbf{G}\mathbf{e}$.

The linear test framework for LPN allows the adversary to process the matrix $\mathbf{A}$ arbitrarily and output a vector $\mathbf{v} \in \mathbb{F}^k$. The adversary wins if the bias of $\mathbf{v} \cdot (\mathbf{A}\mathbf{s} + \mathbf{e})$ is high, where $\cdot$ denotes the inner product. The bias of a distribution $\mathcal{D}$ with respect to a vector $\mathbf{v}$ is denoted $\text{bias}_{\mathbf{v}}(\mathcal{D})$ and measures how closely the distribution and the vector are correlated. The bias of $\mathcal{D}$ is taken to be the maximum bias of $\text{bias}_{\mathbf{v}}(\mathcal{D})$ over all non-zero $\mathbf{v}$. In more detail the definition of the bias is $\text{bias}_{\mathbf{v}}(\mathcal{D}) = |\mathbb{E}_{d \in \mathcal{D}} [\chi(\mathbf{v} \cdot d)]|$, where $\mathbb{E}$ denotes expectation and $\chi : \mathbb{F} \rightarrow \mathbb{C}$ is a non-trivial character.

The analysis of the resistance of DSSD to the linear test framework is identical to the analysis of SSD in [52], Theorem 3, showing that given the same parameters both assumptions are equally immune to this type of attack. The first step is analyzing the analogous LPN instance instead of the DSSD instance. For $r = 1$ if $\mathbf{v}$ is a codeword then the bias is 0, which is identical to the uniform case. If $\mathbf{v}$ is not a codeword then its bias is nonzero only if its nonzero components do not intersect any of the error locations in $\mathbf{e}$. This occurs with low probability if $\mathbf{A}$ generates a code with high minimum distance, or equivalently $\mathbf{G}$ has high dual distance. In DSSD, as is the case for SSD, the bias cannot be increased when $r > 1$.

The last attack we consider is the algebraic attack of [29]. It uses the XL algorithm [35] to generate a large number of polynomial equations that depend on the error vectors. The attack then runs a step of re-linearization and solves the resulting system of equations.

The system of equations is made up of a set of linear parity check equations, and non-linear equations that depend only on the error vectors. The number of the parity check equations is identical for SSD and DSSD and there is no obvious difference (such as the rank weakness) between the two cases. Furthermore, the non-linear equations do not depend on the matrices, and therefore form the same system of equations for SSD and DSSD.

Given the above, we conjecture that the security of DSSD against the attack of [29] is similar to the security of SSD against this attack. Since that attack seems much weaker against SSD than attacks based on the linear test framework, we conjecture that is also the case for DSSD.

D Verifying Π_{SH}^+ using ASL-PIR and zk-FLIOP

In this section we present a second version of Π_{Vrfy} , used for verifying the correct execution of the semi-honest protocol Π_{SH}^+ .

D.1 Fully Linear Proof Systems

A main technical building block in our protocol, which was also used in [27, 28], is a *fully linear* proof system [14], enabling information-theoretic sublinear-communication zero-knowledge proofs on *robustly* secret-shared or distributed input statements. In a nutshell, a *zero-knowledge fully linear interactive oracle proof* (zk-FLIOP) is an information-theoretic proof system in which a prover P wishes to prove some statement about an input x to a verifier V . The protocol takes place in rounds where in each round P produces a proof which, together with x , can be queried by V using *linear queries* only. Then, a public random challenge is generated and the parties proceed to the next round. At the end, the verifier V either accepts or rejects based on the answers it received to its queries.

Definition D.1 (Public-coin zk-FLIOP [14]). *A public-coin fully linear interactive proof system over a ring R with ρ -round and ℓ -query and message length $(u_1, \dots, u_\rho) \in \mathbb{N}^\rho$, consists of a randomized prover algorithm P and a deterministic verifier algorithm V . Let the input to P be $x \in R^n$ and let $r_0 = \perp$. In each round $i \in [\rho]$:*

1. P outputs a proof $\pi_i \in R^{u_i}$, computed as a function of $x, r_1, \dots, r_{i-1}$ and $\pi_1, \dots, \pi_{i-1}$.
2. V samples a random public challenge $r_i \in_R S_i$.
3. ℓ linear oracle queries $q_1^i, \dots, q_\ell^i \in R^{n+u_i}$ are the output of a query generation algorithm $Q_i(r_1, \dots, r_i) : (S_1 \times \dots \times S_i) \rightarrow (R^{n+u_i})^\ell$. Then, V receives ℓ answers $(\langle q_1^i, x || \pi_i \rangle, \dots, \langle q_\ell^i, x || \pi_i \rangle)$.

At the end of round ρ , V outputs **accept** or **reject** based on the random challenges and all the answers to the queries.

Let $\mathcal{L} \subseteq R^n$ be an efficiently recognizable language. We say that ρ -round ℓ -query interactive fully linear protocol $(P_{\text{FLIOP}}, V_{\text{FLIOP}})$ over R is zero-knowledge fully linear interactive oracle proof system for $\mathcal{L}$ with soundness error ϵ if it satisfies the following properties:

- **COMPLETENESS:** If $x \in \mathcal{L}$, then V_{FLIOP} always outputs **accept** when interacting with prover P_{FLIOP} .
- **SOUNDNESS:** If $x \notin \mathcal{L}$, then for all provers P^* , the probability that V_{FLIOP} outputs **accept** is at most $2^{-\epsilon}$.
- **ZERO KNOWLEDGE:** There exists a simulator $\mathcal{S}_{\text{FLIOP}}$ such that for all $x \in \mathcal{L}$ it holds that $\mathcal{S}_{\text{FLIOP}} \equiv \text{view}_{[P_{\text{FLIOP}}(x), V_{\text{FLIOP}}]}(V_{\text{FLIOP}})$ (where the verifier's view $\text{view}_{[P_{\text{FLIOP}}(x), V_{\text{FLIOP}}]}(V_{\text{FLIOP}})$ consists of $\{r_i\}_{i \in [\rho]}$, $\{(q_1^i, \dots, q_\ell^i)\}_{i \in [\rho]}$ and $(\langle q_1^i, x || \pi_i \rangle, \dots, \langle q_\ell^i, x || \pi_i \rangle)_{i \in [\rho]}$).

Remark D.1 (Notation for V_{FLIOP}). For simplicity of notation, in our FLIOP-based constructions in the following sections, we consider only the non-adaptive FLIOP case where all queries of V_{FLIOP}

are being generated and evaluated together as a function of public challenges, as opposed to split per round of the FLIOP with more fine-grained dependence on round challenges thus far and applying only to the proof π_i of the current round. That is, the query generation procedure for all rounds together will be denoted by $\text{FLIOP.QueryGen}(r_1, \dots, r_\rho) = (q_1, \dots, q_\ell) \in (R^{n+u})^\ell$, for $u = \sum_{i=1}^\rho u_i$, resulting in ℓ linear queries, each of which is applied to the concatenation $x || \pi_1 || \dots || \pi_\rho$ of the input x and all round proofs.

Remark D.2 (Query generation algorithms). Our work requires securely computing the FLIOP query vectors from the round challenges by means of secure computation. Towards complexity analyses we denote by Q^π the algorithm that takes as input the round challenges and outputs the query vectors that are applied to the round proofs. Similarly, we denote by Q^x the algorithm that takes as input the round challenges and outputs the query vectors that are applied to the input statement x .

In this paper, we will use this tool for degree- d languages, for a constant value d . That is, languages for which membership can be checked using a degree- d polynomial. The following theorem, which will be used later on, states that for degree- d languages there are zk-FLIOP protocols with sublinear communication and rounds in the size of the input and number of monomials.

Theorem D.1 ([14]). Let $q : R^n \rightarrow R$ be a polynomial of degree- d with M monomials, and let $\mathcal{L}_q = \{x \in R^n \mid q(x) = 0\}$. Let ϵ be the required soundness error. Then, there is a zk-FLIOP for $\mathcal{L}_q$ with the following properties:

- **CONSTANT ROUNDS**, $d = 2$: It has 1 round, proof length $O(\eta\sqrt{n})$, challenge length $O(\eta)$ and the number of queries is $O(\sqrt{n})$, where $\eta = \log_{|R|} \left(\frac{\sqrt{n}}{\epsilon} \right)$ when R is a finite field, and $\eta = \log_2 \left(\frac{\sqrt{n}}{\epsilon} \right)$ when $R = \mathbb{Z}_{2^k}$. The computational complexity is $\tilde{O}(M)$ and the proof generation is a degree-2 function of the input x and the prover's secret randomness, determined by the circuit C and the public randomness.
- **LOGARITHMIC ROUNDS**, $d \geq 2$: It has $O(\log M)$ rounds, proof length $O(d\eta \log M)$, challenge length $O(\eta \log M)$ and the number of queries is $O(d + \log M)$, where $\eta = \log_{|R|} \left(\frac{d \log n}{\epsilon} \right)$ when R is a finite field, and $\eta = \log_2 \left(\frac{d \log n}{\epsilon} \right)$ when $R = \mathbb{Z}_{2^k}$. The computational complexity is $O(dM)$.

Based on the zk-FLIOP constructions above, [14] also show how to construct zero-knowledge proofs over distributed data (DZKPs) where a single prover holds some statement of size n over some field $\mathbb{F}$ that is also additively and robustly shared across multiple verifiers to prove to the parties that their shared statement belongs to some low degree language. In this work we are interested in particular in the *inner-product language* $\mathcal{L}_{IP}^{\mathbb{F},n} = \{\vec{x}, \vec{y}, z \mid \langle \vec{x}, \vec{y} \rangle = z\} \subseteq \mathbb{F}^{2n+1}$. Following from the efficiency of the zk-FLIOP from Theorem D.1, DZKPs with $O(\log n)$ rounds and $O(\log n)$ communication and $O(n)$ computational complexity can be achieved for the inner-product language.

D.2 The verification Protocol

The protocol takes as input the *masked* values $\hat{v}_w$ for each wire w that is either an output wire of the circuit or an input wire to a multiplication gate, and a *succinct* representation of the masks — authenticated shares of the non-zero positions in $\vec{e}_a, \vec{e}_b, \vec{e}_c$ and their values in these non-zero positions. The protocol is parameterized by a zk-FLIOP protocol $\text{FLIOP} = (\text{P}_{\text{FLIOP}}, \text{V}_{\text{FLIOP}})$ for

proving and verifying the correctness of degree-2 inner-product (i.e. a statement of the form $\langle \vec{a}, \vec{b} \rangle = 0$), a field $\mathbb{F}$, a number of multiplication gates n and a statistical security parameter σ

More concretely, we obtain a new version of the FLIOP-based “BGIN compiler” [27, 28] for m parties to compute an arithmetic circuit C of size n (i.e., number of multiplication gates), with malicious security. The main feature of our protocol as compared with prior works [27, 28] is that the verification communication overhead (beyond the semi-honest protocol) is $m \log(n)$, the size of the correlated randomness (beyond the semi-honest triples) is $O(\log(n))$ and the computational work per party is $O(n)$. This is better than [27] where the size of the correlated randomness is $O(m \cdot \log(n))$ and computational work is $O(m \cdot n)$ and better than [28] where the communication overhead is $O(m \cdot \sqrt{n})$ and the size of the correlated randomness is $O(\sqrt{n})$.

As in our verification protocol in Section 4.2, the first step is to reduce the verification goal to checking that an inner product between $\vec{a}$ and $\vec{b}$ equals 0 (see Eq. (4) and (5)). It follows that the equation to verify is an instance of the inner-product language $\mathcal{L}_{IP}^{\mathbb{F}, 3n}$ over the inputs $\vec{a}, \vec{b}, \Lambda$. Thus, by Theorem D.1, there exists a zk-FLIOP to prove that the equation holds, where the proof’s size is sublinear in the size of the input.

Now, there are two main challenges before using the zk-FLIOP machinery. First, the statement to be proven is not known to any of the parties, and so it is not clear how the zk-FLIOP prover will be emulated. In the solution of [28] a “distributed prover” approach was taken (first used in [14]), where all parties emulate together the role of a single prover, and all parties emulate the role of the zk-FLIOP verifier. A second challenge is that if all parties emulate the zk-FLIOP verifiers, then it is not clear how to prevent cheating (and maintain the soundness property of the zk-FLIOP). To solve this, prior works showed that it is possible to precompute the zk-FLIOP queries’ answers over the vector $\vec{b}'$ and let the parties hold an authenticating secret sharing of this data. When the zk-FLIOP verifier is emulated, the parties reconstruct this data. Since it is shared in an authenticated manner, it prevents corrupted parties from making the verification protocol accept false statements (which correspond to cheating in the semi-honest circuit computation). Moreover, since the number of queries is sublinear in the circuit size, this precomputed correlated randomness (referred to as “FLIOP correlations”) requires a succinct amount of storage.

However, using the distributed prover approach, [28] were unable to emulate a zk-FLIOP with a logarithmic number of rounds, since the parties need to generate a proof in each round without interaction (otherwise, communication would not be sublinear). This was achievable in their case using the single-round FLIOP of [14], which had the special property that the Prover algorithm for a degree-2 statement is computable as a (compressing) degree-2 function of the statement itself; thus, given the “star-sharing” of the relevant values (supporting a multiplicative property), parties were able to locally compute additive shares of the prover output without interaction. However, the multi-round FLIOP requires more complex computations, which lose this crucial degree-2 property. As a result, [28] was stuck at the $O(\sqrt{n})$ proof size of the 1-round FLIOP, appearing in their correlation size and offline communication.

Our approach. Our protocol improves upon previous works in two ways. First, we show how to make the distributed prover approach work over many rounds of FLIOP interaction, thereby reducing communication and computational work. Second, we show how to prepare the query responses over the triples (the vector b) online, instead of in advance, yielding significant performance gains. Intuitively, this approach is better since the query responses are computed only after the zk-FLIOP challenges are known. As we will see, this enables us to reduce the problem of computing

the responses to a sparse linear private information retrieval over a public database, computed from the random challenges. As for the first improvement, we observe that the multi-round FLIOP construction in [14] satisfies the following “prover degree preservation” property:

Lemma D.1 (prover degree preservation). *The prover’s algorithm in each round of the FLIOP construction of [14] for the inner-product language $\mathcal{L}_{IP}^{\mathbb{F},n}$ is linear in the vector $\vec{b}$ and the prover’s randomness.*

Proof. In the construction of [14], the prover computes its proof in round j as a function of the input $\vec{a}, \vec{b}$, the random challenges $c_1, \dots, c_{j-1}$ and its own randomness. In the first round, the proof is computed by splitting $\vec{a}$ and $\vec{b}$ to equally-sized vectors $\vec{a}_1, \vec{a}_2$ and $\vec{b}_1, \vec{b}_2$. The proof consists of three elements: $\langle \vec{a}_1, \vec{b}_1 \rangle$, $\langle \vec{a}_2, \vec{b}_2 \rangle$ and $\langle \vec{a}_3, \vec{b}_3 \rangle$, where $\vec{a}_3, \vec{b}_3$ are computed as a public linear combination of $\vec{a}_1, \vec{a}_2$ and $\vec{b}_1, \vec{b}_2$ respectively. Hence, the proof satisfies the claimed property. Next, in round $j > 1$, the proof is also computed as a function of previous round challenges. Specifically, first the prover applies an affine function that is computed from the challenges $c_1, \dots, c_{j-1}$ on $\vec{a}, \vec{b}$ to obtain $\vec{a}'$ and $\vec{b}'$ of smaller size. The proof is then computed as in the first round on $\vec{a}'$ and $\vec{b}'$. Hence, again, the proof is a linear function in $\vec{b}$. In the last round, the only change is that the randomness of the prover is also used, to achieve zero-knowledge. The prover chooses $\vec{a}_0''$ and $\vec{b}_0''$, and then the proof consists of $\vec{a}_0'', \vec{b}_0''$ and the elements $\langle \vec{a}_0'', \vec{b}_0'' \rangle$, $\langle \vec{a}_1'', \vec{b}_1'' \rangle$, $\langle \vec{a}_2'', \vec{b}_2'' \rangle$, $\langle \vec{a}_3'', \vec{b}_3'' \rangle$, $\langle \vec{a}_4'', \vec{b}_4'' \rangle$ where $\vec{a}_1'', \dots, \vec{a}_4''$ and $\vec{b}_1'', \dots, \vec{b}_4''$ are computed as an affine function over $\vec{a}$ and $\vec{b}$ as before. Hence, the claim holds for all rounds. $\square$

Recall that the vector $\vec{b}$ corresponds to the triples payload vector, of which the parties hold additive shares. This property thus enables the parties to compute additive shares of the proof, by applying the corresponding linear map on their shares of $\vec{b}$.

In the protocol, the parties emulate the prover in each round by computing a share of the proof and secret sharing it to the other parties in a robust way. Specifically, each party “star-shares” its share of the proof to the other parties, i.e., sends a mask of its share to the other parties, where the mask is shared across the parties. Then, the parties sample the random challenge and compute the FLIOP queries’ answers over the public vector $\vec{a}$. For the shared vector $\vec{b}$, the parties call the $\mathcal{F}_{PIR}^{ASL}$ functionality to perform a sparse secret linear combination over a public database computed from the public queries and the public compressive mappings G_A, G_B, G_c used to produce the random triples. The linear combination corresponds to the non-zero positions in the sparse vectors $\vec{e}_a, \vec{e}_b, \vec{e}_c$. The formal description appears in Protocol D.2.

Security. Security of the resulting protocol follows from the same analysis of Theorem 4.2. In a nutshell, simulation of the protocol is enabled by the fact that the adversary sees random masks that are distributed uniformly and the zero-knowledge property which says that the queries’ answers can be simulated. Hence, nothing can be learned from the protocol. On the other hand, if cheating took place in the semi-honest protocol, then the parties output **accept** only if the random linear combination yields 0, or if the zk-FLIOP emulation output **accept**, which happens with small probability.

Protocol D.2 (Π_{Vrfy}). *A Protocol for Verifying the correct execution of Π_{SH}^+*

Parameters:

- m parties $P_1, \dots, P_m$, input size $n \in \mathbb{N}$, a field $\mathbb{F}$ and an extension field $\hat{\mathbb{F}}$.
- A ρ -round and ℓ -query and message length $(u_1, \dots, u_\rho) \in \mathbb{N}^\rho$ for the language $\mathcal{L}_{IP}^{\mathbb{F}, 6n+1}$ zk-FLIOP protocol $(P_{\text{FLIOP}}, V_{\text{FLIOP}})$ over $\hat{\mathbb{F}}$.
- Sparse vector sizes N, N' and compressive linear mappings $G_A, G_B \in \mathbb{F}^{n \times N}$ and $G_C \in \mathbb{F}^{n \times N'}$.
- A circuit C over a field $\mathbb{F}$ with n multiplication gates and o output wires.
Let T be the set of multiplication gates' input and output wires.
Let O be the set of output wires.

Input: The parties hold:

- For each wire $w \in T \cup O$: $\hat{v}_w = v_w + r_w$ where r_w is a random mask
- $\llbracket \vec{r}_T \rrbracket$ where $\vec{r}_T = (r_w)_{w \in T}$ (i.e., the vector of all multiplication gate wire payloads).
- $\langle\langle \vec{r}_O \rangle\rangle$ where $\vec{r}_O = (r_w)_{w \in O}$ (i.e., the vector of all circuit output wire payloads).
- $\langle\langle \vec{I}_\gamma \rangle\rangle, \langle\langle \vec{V}_\gamma \rangle\rangle$ for $\gamma \in \{A, B, C\}$, which are authenticated shares of the non-zero positions in $\vec{e}_a, \vec{e}_b, \vec{e}_c$ and their values in these positions (obtained from $\mathcal{F}_{\text{Triples}}$).

The protocol: Note: All computations in the protocol are carried out over $\hat{\mathbb{F}}$.

1. Preparing $\vec{a}, \Lambda$: the parties call $\mathcal{F}_{\text{Coin}}$ to receive a random coefficient $\alpha_w \in \hat{\mathbb{F}}$ for each wire $w \in T \cup O$, and locally compute $\vec{a}$ and Λ as defined in the text.
Let $\vec{a}_T$ be the first $3n$ elements in $\vec{a}$ and let $\vec{a}_O$ be the last o elements in $\vec{a}$.
2. The parties define $\llbracket \vec{b}_T \rrbracket = \llbracket \vec{r}_T \rrbracket$ and $\langle\langle \vec{b}_O \rangle\rangle = \langle\langle \vec{r}_O \rangle\rangle$.
3. For each round j of the zk-FLIOP and each party $i \in [m]$:
 - (a) The parties call $\mathcal{F}_{\text{Rand}}$ to receive $\langle\langle \tilde{\pi}_j \rangle\rangle$ where $\tilde{\pi}_j \in \hat{\mathbb{F}}^{u_j}$.
 - (b) Each party P_i computes its share $\tilde{\pi}_{i,j} \in \hat{\mathbb{F}}^{u_j}$ of the round proof as a function of $\vec{a}, \llbracket \vec{b} \rrbracket$ and $c_1, \dots, c_{j-1}$ (previous round challenges) and sends $\hat{\pi}_{i,j} = \tilde{\pi}_{i,j} + \tilde{\pi}_{i,j}$ to the other parties.
 - (c) The parties locally compute $\hat{\pi}_j = \sum_{i=1}^m \hat{\pi}_{i,j}$.
 - (d) The parties call $\mathcal{F}_{\text{Coin}}$ to receive a random challenge c_j .
4. Computing the queries: the parties locally compute $Q = \text{FLIOP.QueryGen}(c_1, \dots, c_\rho)$. Note that Q has ℓ rows (number of queries).
5. Computing the queries' answers:
 - (a) The parties locally compute

$$\hat{e}_1, \dots, \hat{e}_\ell \leftarrow \langle \vec{Q}[1, \cdot], \Lambda \|\vec{a}\| 0^{3n} \|\hat{\pi}_1\| \cdots \|\hat{\pi}_\rho\rangle, \dots, \langle \vec{Q}[\ell, \cdot], \Lambda \|\vec{a}\| 0^{3n} \|\hat{\pi}_1\| \cdots \|\hat{\pi}_\rho\rangle$$

- (b) Let G be a matrix where G_A, G_B, G_C are positioned along its diagonal (Note that $G \in \hat{\mathbb{F}}^{3n \times (2N+N')}$).

Let $Q = Q_\Lambda || Q_a || Q_O || Q_T || Q_\pi$, where $Q_\Lambda \in \hat{\mathbb{F}}^\ell, Q_a \in \hat{\mathbb{F}}^{3n \times o}, Q_O \in \hat{\mathbb{F}}^{\ell \times o}, Q_T \in \hat{\mathbb{F}}^{\ell \times 3n}$ and $Q_\pi \in \hat{\mathbb{F}}^{\ell \times \sum_{j=1}^p u_j}$.

The parties compute a public database $DB = Q_T \cdot G$ and send $DB, \langle\langle \vec{I}_A \rangle\rangle || \langle\langle \vec{I}_B \rangle\rangle + n || \langle\langle \vec{I}_C \rangle\rangle + 2n$, and $\langle\langle \vec{V}_A \rangle\rangle || \langle\langle \vec{V}_B \rangle\rangle || \langle\langle \vec{V}_C \rangle\rangle$ to $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ to receive back $e_{1,t}, \dots, e_{\ell,t}$.

- (c) For each $k \in [\ell]$, the parties compute:

$$\langle\langle \tilde{e}_k \rangle\rangle = \langle\vec{Q}_O[k, \cdot], \langle\vec{r}_O \rangle\rangle + e_{k,t} + \langle\vec{Q}_\pi[k, \cdot], \langle\vec{\pi} \rangle\rangle \cdots \langle\langle \vec{\mathcal{T}} \rangle\rangle$$

- (d) For each $k \in [\ell]$, the parties run $\text{Rec}_{\text{Auth}}(\langle\langle \tilde{e}_k \rangle\rangle)$ to receive $\tilde{e}_k$.

- (e) For each $k \in [\ell]$, the parties compute $e_k = \hat{e}_k + \tilde{e}_k$.

6. The parties run the decision predicate of $\mathbf{V}_{\text{FLIOP}}$ on all the queries' answers $e_1, \dots, e_\ell$. If any party receives **reject**, then it outputs **reject**. Otherwise, it outputs **accept**.

Remark D.3. While the use of FLIOPs has been introduced only when relying on DSSD, the same FLIOP-based approach could have been used in $\Pi_{\text{Vrfy}}^{q, \text{SSD}}$ and even in Π_{Vrfy} . However, the main benefit of using FLIOPs is to eliminate the database dependency on $\vec{a}_T$, which is not needed when relying on SSD, while incurring a mild increase in verification complexity and making the presentation more involved.

E Realizing $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ from $\mathcal{F}_{\text{UV}}$

In this section we present a simple protocol for realizing the $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ functionality in the $(\mathcal{F}_{\text{UV}}, \mathcal{F}_{\text{Mul}})$ -hybrid model, denoted by $\Pi_{\text{PIR}}^{\text{UV}}$. While this functionality can be realized from any protocol that securely realizes a natural, maliciously secure PIR functionality, we focus on $\Pi_{\text{PIR}}^{\text{UV}}$, a specific instantiation in the $(\mathcal{F}_{\text{UV}}, \mathcal{F}_{\text{Mul}})$ -hybrid model, given in Protocol E.1. For database size M , batching parameter q and query weight w the protocol takes as input w query positions $\alpha_i \in [M]$ and w query values $\beta_i \in \mathbb{F}^q$ and outputs an authenticated secret sharing of the linear combinations $\sum_{i=1}^w \beta_i[j] DB[\alpha_i]$ for each $j \in [q]$. The authenticated shares of all β_i 's are under the same MAC authentication key Δ , shared among the parties before the protocol. Recall that the authentication of α_i and β_i is $\Delta\alpha_i$ and $\Delta\beta_i$, respectively. The protocol works by calling $\mathcal{F}_{\text{UV}}(\langle\langle \alpha_i \rangle\rangle, (1, \Delta))$ for each $i \in [w]$ from which the parties receive additive shares of the scaled unit vector $\vec{v}_i$ with $\vec{v}_i[\alpha_i] = (1, \Delta)$ and $\vec{v}_i[j] = (0, 0)$ for $j \neq \alpha_i$. Next, by computing the inner product of the database with each of the vectors $\vec{v}_i$ the parties obtain additive shares of the corresponding database entry $DB[\alpha_i]$ along with its authentication $\Delta DB[\alpha_i]$, comprising an authenticated secret sharing of $\langle\langle DB[\alpha_i] \rangle\rangle$. To finish the protocol, parties can securely compute the inner product of the vector $A = (\langle\langle DB[\alpha_1] \rangle\rangle, \dots, \langle\langle DB[\alpha_w] \rangle\rangle)$ with $B_j = (\langle\langle \beta_1[j] \rangle\rangle, \dots, \langle\langle \beta_w[j] \rangle\rangle)$ for each $j \in [q]$ using wq calls to $\mathcal{F}_{\text{Mul}}$. Finally, the parties output an authenticated sharing of the inner product $\langle\langle A, B_j \rangle\rangle$ for each $j \in [q]$, as required by $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$.

Theorem E.2 (Security of $\Pi_{\text{PIR}}^{\text{UV}}$). *Protocol E.1 securely realizes $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ in the $(\mathcal{F}_{\text{UV}}, \mathcal{F}_{\text{Mul}})$ -hybrid model.*

Proof. Correctness follows from the correctness of $\mathcal{F}_{UV}$ and $\mathcal{F}_{Mul}$. Given those the correctness of the computation is trivial. The protocol is silent besides calls to $\mathcal{F}_{UV}$ and $\mathcal{F}_{Mul}$ so by invoking their simulators we get a simulator for the protocol. $\square$

Theorem E.3 (Efficiency of Π_{PIR}^{UV}). *Protocol E.1 makes w calls to $\mathcal{F}_{UV}$, w calls to $\mathcal{F}_{Mul}$ and has local computation cost of $2wM$ field operations.*

F Proof of Theorem 4.2

Proof. Let Sim be an ideal world simulator and let $\mathcal{A}$ be the real-world adversary. The simulator Sim plays the role of the honest parties and ideal functionalities $\mathcal{F}_{Triples}$, $\mathcal{F}_{PIR}^{ASL}$, $\mathcal{F}_{Rand}$, and $\mathcal{F}_{Coin}$, handing $\mathcal{A}$ random shares and coins. Hence, Sim knows all the random masks and coins used by $\mathcal{A}$ in the execution. The simulator works as follows:

- **Input sharing:** For inputs held by honest parties, Sim sends random elements. For each input held by a corrupted party on wire w , Sim receives $\hat{x}_w = x_w + r_w$ and thus can extract $x_w = \hat{x}_w - r_w$.
- **Circuit evaluation:** This step is simulated by sending random values in $\mathbb{F}$ to $\mathcal{A}$ as the honest parties' messages. As shown in [27] (see Appendix A), the semi-honest protocol is secure up to additive attack, which means that the simulator can extract the additive error ϵ_w for each wire that is an input to a multiplication gate or a circuit output wire. Sim thus records all the masked values on each wire and all the additive errors. In addition, Sim computes the corrupted parties' shares on each output wire of the circuit.
- **Verification:** The simulator Sim samples the random coefficients α_w and hands them to the parties. If there exists w such that $\epsilon_w \neq 0$, but still $\sum_{w \in W} \alpha_w \cdot \epsilon_w = 0$, then Sim outputs **fail** and halts. Otherwise, Sim computes $\vec{a}$, Λ and the corrupted parties' shares of $\langle \vec{a}_O, \vec{b}_O \rangle$. Then, playing the role of $\mathcal{F}_{PIR}^{ASL}$, Sim hands $\mathcal{A}$ random shares as the corrupted parties' shares of $\langle e \rangle$ and computes the corrupted parties' shares in $\langle u \rangle$. Finally, Sim chooses random shares for the honest parties as their shares of u , under the constraint that an honest reconstruction will yield $\sum_{w \in W} \alpha_w \cdot \epsilon_w$. Then, it simulates the execution of $\text{Rec}_{Auth}(\langle u \rangle)$ by playing the role of the honest parties. If the reconstruction fails or $u \neq 0$, Sim sends **abort** to the trusted party computing f , simulates the honest parties aborting in the real execution, and outputs whatever $\mathcal{A}$ outputs.
- **Output reconstruction:** Sim sends **continue** to the trusted party to receive back the corrupted parties outputs. For each wire w with output intended to a corrupted party, let $\hat{x}_w$ be the masked value on the wire and x_w be the output received from the trusted party. Sim chooses random shares for the honest parties for the mask r_w , under the constraint that the reconstruction will yield $r_w = \hat{x}_w - x_w$. Then, it simulates the share reconstruction on all output wires by playing the role of the honest parties. If the reconstruction failed on a wire with an output intended to an honest party P_j , then Sim sends **abort_j** to the trusted party computing f . Finally, Sim outputs whatever $\mathcal{A}$ outputs.

Observe that there is no difference between the simulated and real execution except for the event where Sim outputs **fail**. This holds since the adversary sees only random masked values and random shares throughout both executions. Note that the **fail** event happens when cheating takes place, but the honest parties output **accept** at the end of real-world verification since the random linear combination yields 0. This event thus happens with probability $\frac{1}{\mathbb{F}}$, which is allowed by the theorem.



Protocol 4.1 (Π_{Vrfy}). *A Protocol for Verifying the Correct Execution of Π_{SH}^+*

Parameters:

- m parties $P_1, \dots, P_m$, input size $n \in \mathbb{N}$, a field $\mathbb{F}$ and an extension field $\hat{\mathbb{F}}$.
- Sparse vector sizes N, N' and compressive linear mappings $G_A, G_B \in \mathbb{F}^{n \times N}$ and $G_C \in \mathbb{F}^{n \times N'}$
- A circuit C over a field $\mathbb{F}$ with n multiplication gates and o output wires.
Denote by T the set of multiplication gates' input and output wires.
Denote by O the set of output wires.

Input: The parties hold:

- For each wire $w \in T \cup O$: $\hat{v}_w = v_w + r_w$ where r_w is a random mask
- $\llbracket \vec{r}_T \rrbracket$ where $\vec{r}_T = (r_w)_{w \in T}$ (i.e., the vector of all multiplication gate wire payloads).
- $\langle\langle \vec{r}_O \rangle\rangle$ where $\vec{r}_O = (r_w)_{w \in O}$ (i.e., the vector of all circuit output wire payloads).
- $\langle\langle \vec{I}_\gamma \rangle\rangle, \langle\langle \vec{V}_\gamma \rangle\rangle$ for $\gamma \in \{A, B, C\}$, which are authenticated shares of the non-zero positions in $\vec{e}_a, \vec{e}_b, \vec{e}_c$ and their values in these positions (obtained from $\mathcal{F}_{\text{Triples}}$).

The protocol: Note: All computations in the protocol are carried out over $\hat{\mathbb{F}}$.

1. Preparing $\vec{a}$, Λ : the parties call $\mathcal{F}_{\text{Coin}}$ to receive a random coefficient $\alpha_w \in \hat{\mathbb{F}}$ for each wire $w \in T \cup O$ and compute $\vec{a}$ and Λ as defined in the text.
Let $\vec{a}_T$ be the first $3n$ elements in $\vec{a}$ and let $\vec{a}_O$ be the last o elements in $\vec{a}$.
 2. The parties define $\llbracket \vec{b}_T \rrbracket = \llbracket \vec{r}_T \rrbracket$ and $\langle\langle \vec{b}_O \rangle\rangle = \langle\langle \vec{r}_O \rangle\rangle$.
 3. Let G be a matrix where G_A, G_B, G_C are positioned along its diagonal (note that $G \in \mathbb{F}^{3n \times (2N + N')}$).
The parties compute a public database $\vec{D}\vec{B}$ of size $2N + N'$ where $\vec{D}\vec{B}[i] = \langle \vec{a}_T, G \cdot \text{unit}(i) \rangle$ and send $\vec{D}\vec{B}, \langle\langle \vec{I}_A \rangle\rangle || (\langle\langle \vec{I}_B \rangle\rangle + N) || (\langle\langle \vec{I}_C \rangle\rangle + 2N)$, and $\langle\langle \vec{V}_A \rangle\rangle || \langle\langle \vec{V}_B \rangle\rangle || \langle\langle \vec{V}_C \rangle\rangle$ to $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ to receive back $\langle\langle e \rangle\rangle$.
 4. The parties compute $\langle\langle u \rangle\rangle = \Gamma + \langle \vec{a}_O, \langle\langle \vec{b}_O \rangle\rangle \rangle + \langle\langle e \rangle\rangle$.
 5. The parties run $\text{Rec}_{\text{Auth}}(\langle\langle u \rangle\rangle)$ to receive back u and check that $u = 0$. If it holds, they output **accept**. Otherwise, they output **reject**.
-

Protocol 5.1 ($\Pi_{\text{PIR}}^{\text{DMPF}}$). *Realizing $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ from $\mathcal{F}_{\text{DMPF}}$*

Parameters: Database $DB \in \mathbb{F}^{M \times q}$ of size M , number of parties m , query weight w , batching parameter $q \in \mathbb{N}$.

Inputs: $\langle\langle \alpha_i \rangle\rangle, \langle\langle \beta_i \rangle\rangle$ for $i \in [w]$.

Output: $\langle\langle \sum_{i=1}^w \beta_i DB[\alpha_i] \rangle\rangle_{\mathbb{F}^q}$.

1. View each authenticated sharing of β_i as a pair $\langle\langle \beta_i \rangle\rangle = (\beta_i, \Delta\beta_i)$ for $i \in [w]$ for authentication key $\Delta \in \hat{\mathbb{F}}$ which authenticates all β_i 's.
 2. Call $\mathcal{F}_{\text{DMPF}}(\{\langle\langle \alpha_i \rangle\rangle, \langle\langle \beta_i \rangle\rangle\}_{i \in [w]})$ to receive additive shares of the sparse vector $\vec{v} \in \left(\left(\mathbb{F} \times \hat{\mathbb{F}}\right)^q\right)^M$.
 3. View $\vec{v}$ as a matrix in $\mathbb{F}^{M \times q}$ where $\vec{v}[j]$ is the j th row of the matrix.
 4. Output m -party authenticated sharing $\langle\langle \sum_{i=1}^w \vec{v}[\alpha_i] \odot DB_{\alpha_i, \cdot} \rangle\rangle$.
-

Protocol E.1 ($\Pi_{\text{PIR}}^{\text{UV}}$). *Realizing $\mathcal{F}_{\text{PIR}}^{\text{ASL}}$ from $\mathcal{F}_{\text{UV}}$*

Parameters: Database $DB \in \mathbb{F}^M$ of size M , number of parties m , query weight w , batching parameter $q \in \mathbb{N}$.

Inputs: $\langle\langle \alpha_i \rangle\rangle, \langle\langle \beta_i \rangle\rangle$ for $i \in [w]$ where $\alpha_i \in [M]$ and $\beta_i \in \mathbb{F}^q$.

Output: $\langle\langle \sum_{i=1}^w \beta_i DB[\alpha_i] \rangle\rangle_{\mathbb{F}}$.

1. For each $i \in [w]$ call $\mathcal{F}_{\text{UV}}(\langle\langle \alpha_i \rangle\rangle, \langle\langle 1_{\mathbb{F}} \rangle\rangle)$ to receive additive shares of the scaled unit vector $\vec{v}_i \in \mathbb{F}^M$ with $\vec{v}_i[\alpha_i] = (1, \Delta)$ and $\vec{v}_i[j] = (0, 0)$ for $j \neq \alpha_i$.
 2. For each $i \in [w]$ locally compute an authenticated secret sharing of the database entry at position α_i as follows:
 - (a) Locally compute additive shares of the inner product $A_i = \langle DB, \vec{v}_i \rangle$.
 - (b) Set $\langle\langle DB[\alpha_i] \rangle\rangle = A_i$.
 3. Locally compute an authenticated secret sharing of the linear combination of the queried entries as follows:
 - (a) For each $i \in [w]$ call $\mathcal{F}_{\text{Mul}}(\langle\langle DB[\alpha_i] \rangle\rangle, \langle\langle \beta_i \rangle\rangle)$ to receive an authenticated secret sharing of $\langle\langle \beta_i DB[\alpha_i] \rangle\rangle$.
 - (b) Compute $\langle\langle \sum_{i=1}^w \beta_i DB[\alpha_i] \rangle\rangle$.
 4. $\langle\langle \sum_{i=1}^w \beta_i DB[\alpha_i] \rangle\rangle_{\mathbb{F}}$
-